

DSCC

Distributed Systems and Cloud Computing — Project Report



Cloud-Based Campus Placement Management System

Submitted by

Kaif Khan — Roll No: B-19

Richa Maurya — Roll No: B-29

Vrushket Mulye — Roll No: 33

Under the Guidance of

Dr. Nidhi Punia

Live Project: <https://65.1.71.208>

GitHub Repository: github.com/Kaifkhan1212/CloudPMS-PlacementPortal

Table of Contents

1. Executive Summary & Project Overview	3
1.1 Problem Statement.....	3
1.2 Key Objectives.....	3
1.3 Project at a Glance	3
2. Cloud Architecture	4
2.1 Layers of the Architecture.....	4
3. Distributed Systems & Cloud Computing Concepts Applied	5
4. Deep-Dive: AWS & Cloud Services Utilised.....	6
4.1 Amazon EC2 (Elastic Compute Cloud)	6
4.2 Amazon S3 (Simple Storage Service).....	6
4.3 Amazon Textract	7
4.4 Amazon SES (Simple Email Service)	7
4.5 AWS IAM (Identity and Access Management)	7
4.6 MongoDB Atlas (Database as a Service)	8
4.7 DNS, Nginx, PM2 and HTTPS	8
5. Technology Stack.....	9
6. Functional Modules & User Roles.....	10
6.1 User Roles	10
6.2 Modules	10
6.3 Application Status Lifecycle	11
6.4 Eligibility Rule	11
7. Database Design.....	11
7.1 User	11
7.2 StudentProfile (one-to-one with User)	11
7.3 Drive.....	12
7.4 Application	12
7.5 EmailLog	12
8. Authentication, Authorisation & Security	13
8.1 JWT Strategy: Stateless + Stateful Hybrid	13
8.2 Role-Based Access Control.....	13
8.3 Security Measures.....	13
9. API Reference & Processing Flows.....	14
9.1 REST API Summary	14
9.2 Resume Upload and Processing Flow	15
9.3 SES Notification Flow	15
10. Environment Configuration Reference (.env)	15
11. Production Deployment & Setup — Screenshots.....	16
11.1 Amazon S3 Bucket.....	16
11.2 IAM User and Access.....	18
11.3 Amazon SES Setup & Identity Verification	20
11.4 Amazon EC2 Instance.....	23
11.5 Security Group Configuration	25
11.6 Connecting to the Server	26
11.7 Server Preparation: Packages and Node.js	27
11.8 Cloning the Repository & Environment Configuration.....	28
11.9 Starting the Backend & Health Check	29
11.10 PM2 Process Management	30
11.11 Building the React Frontend	30

11.12 HTTPS with Let's Encrypt.....	31
12. Live Application Walkthrough	32
12.1 Login and Registration	32
12.2 Student Workflow	32
12.3 Placement Cell Workflow	33
12.4 Administrator Workflow	34
13. Live Deployment & Repository Links	34
13.1 Deployment Runbook	35
14. Testing, Results, Limitations & Future Scope.....	35
14.1 Testing.....	35
14.2 Results.....	35
14.3 Limitations	35
14.4 Future Scope	36
15. Conclusion.....	36
References	36

1. Executive Summary & Project Overview

CloudPMS is a cloud-based Campus Placement Management System that centralises student academic profiles, placement drives, eligibility screening, application tracking, resume storage and placement notifications on one platform. It is built on a modern JavaScript stack (React 19, Vite, Tailwind CSS, Node.js, Express 4 and MongoDB) and is deployed on Amazon Web Services (AWS) in the Asia Pacific (Mumbai) `ap-south-1` region.

Three roles use the system: **Students** maintain profiles, upload resumes and apply to drives; **Placement Cell officers** create drives, manage applicant pipelines, schedule interviews and send communication; **Administrators** govern users and view placement reports. Access is enforced by JWT authentication and role-based access control.

1.1 Problem Statement

Campus placements are commonly run with spreadsheets, e-mail threads and notice boards. This causes the following problems:

- No central repository for student profiles and resumes.
- Manual, time-consuming and error-prone eligibility screening against company criteria.
- Delayed or inconsistent status communication to students.
- No audit trail of how an application progressed.

1.2 Key Objectives

- **Centralised student profiles and cloud resume storage:** academic data (branch, CGPA, backlogs, 10th/12th percentages) and PDF resumes stored privately in Amazon S3.
- **Automated eligibility filtering:** students see only the drives they qualify for, based on branch, minimum CGPA and maximum backlogs.
- **Application lifecycle tracking:** Applied → Shortlisted → Interview Scheduled → Selected / Rejected, with remarks and interview dates.
- **AI-assisted skill extraction:** Amazon Textract reads each uploaded resume and detected skills are saved to the student profile.
- **Automated and manual e-mail notifications:** Amazon SES delivers confirmations, status updates and Communication Center messages; every e-mail is logged.
- **Secure-by-design cloud deployment:** IAM instance role (no static keys on the server), private S3 bucket with time-limited presigned URLs, Nginx reverse proxy with HTTPS (Let's Encrypt), PM2 process supervision.

1.3 Project at a Glance

Item	Details
Project title	CloudPMS — Cloud-Based Campus Placement Management System
Subject	Distributed Systems and Cloud Computing (DSCC)
Guide	Dr. Nidhi Punia
Cloud provider / region	Amazon Web Services — Asia Pacific (Mumbai) <code>ap-south-1</code>
Compute	Amazon EC2 t3.micro, Ubuntu Linux, Elastic IP 65.1.71.208

Item	Details
Managed services used	Amazon S3, Amazon Textract, Amazon SES, AWS IAM, MongoDB Atlas
Frontend / Backend	React 19 + Vite 8 + Tailwind CSS 3 / Node.js 22 + Express 4 + Mongoose 8

2. Cloud Architecture

CloudPMS follows a **three-tier architecture** hosted on a single EC2 instance, with all stateful and AI/e-mail capabilities delegated to managed cloud services. The request flow is:

1. A user opens `cloudpms.kaifkhan.in`; Hostinger DNS resolves the name to the AWS Elastic IP `65.1.71.208`.
2. The request reaches **Nginx** on the EC2 instance over HTTPS (port 443; port 80 is also open in the security group).
3. For `/api/*`, Nginx proxies the request to the **Node.js/Express** API on `127.0.0.1:5000`; all other paths return the compiled **React** single-page application.
4. Express authenticates the caller (JWT) and authorises the role, then reads/writes data in **MongoDB Atlas** through Mongoose.
5. Resumes are written to a private **Amazon S3** bucket; **Amazon Textract** analyses the stored object and the detected skills are saved to MongoDB.
6. Status changes and manual messages are sent through **Amazon SES**; each e-mail (sent or failed) is recorded in the `EmailLog` collection.
7. The AWS SDK obtains temporary credentials from the EC2 **IAM role** `CloudPMS-EC2-Role` through the instance metadata service (IMDSv2); no access keys are stored on the server.

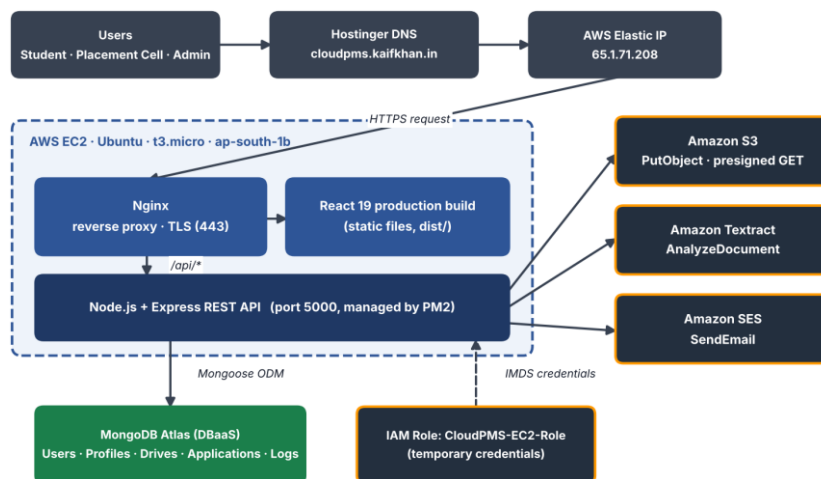


Fig 2.1 — CloudPMS cloud architecture.

2.1 Layers of the Architecture

Layer	Components
Presentation tier	React 19 SPA built with Vite and Tailwind CSS (light/dark themes), served as static files by Nginx
Application tier	Node.js 22 + Express 4 REST API on port 5000 (PM2); JWT auth, RBAC middleware, validation, Helmet, rate limiting
Data tier	MongoDB Atlas (users, student profiles, drives, applications, e-mail logs) and Amazon S3 (resume PDFs)
AI & messaging services	Amazon Textract (resume analysis), Amazon SES (transactional e-mail)
Identity & security	AWS IAM role/instance profile, EC2 security group, Let's Encrypt TLS certificate, IMDSv2

Layer	Components
Network & delivery	Hostinger DNS (A record), AWS Elastic IP, Nginx reverse proxy

3. Distributed Systems & Cloud Computing Concepts Applied

The project is a practical demonstration of several concepts from the DSCC syllabus. The table maps each concept to its concrete implementation in CloudPMS.

Concept	Application in CloudPMS
Client–server model	Browser-based React client communicating with a REST API over HTTPS
Multi-tier architecture	Presentation (React/Nginx), application (Express) and data (MongoDB Atlas, S3) tiers with clear responsibilities
IaaS	Amazon EC2 provides the virtual machine; the OS, Nginx, Node.js and PM2 are configured by the team
DBaaS	MongoDB Atlas runs the database cluster; no database server is installed or patched on EC2
Object storage	Amazon S3 stores resumes with 99.999999999% durability and no capacity planning
Managed AI service	Amazon Textract performs document analysis through an API call; no ML model is trained or hosted
Managed messaging	Amazon SES delivers e-mail without running a mail server
Stateless services	Access tokens are self-contained JWTs, so API instances hold no session state and can be replicated behind a load balancer
Access & location transparency	Users reach the system through a domain name; the Elastic IP hides instance restarts; S3 and Atlas are addressed by name, not by server
Fault tolerance & graceful degradation	PM2 restarts crashed processes and survives reboots; Textract or SES failures never block uploads or status updates; S3 and Atlas replicate data internally
Consistency & integrity	Unique compound index on {student, drive} prevents duplicate applications even under concurrent requests
Security in the cloud	IAM least privilege, role credentials via IMDSv2, private S3 bucket, presigned URLs, TLS, security-group firewalling
Latency & data locality	Compute, storage, AI and e-mail all run in Mumbai (ap-south-1), close to Indian users
Elasticity (design readiness)	Stateless API plus managed services allow a later move to an Application Load Balancer and Auto Scaling group

4. Deep-Dive: AWS & Cloud Services Utilised

Service	Core Purpose	Integration Details
Amazon EC2	Application hosting	Ubuntu t3.micro instance running Nginx, Node.js/Express and PM2
Elastic IP	Static public address	65.1.71.208 mapped by the DNS A record
Amazon S3	Resume storage	Private bucket <code>cloudpms-resumes-kaif-2026</code> ; keys <code>resumes/<userId>/<timestamp>-<file>.pdf</code>
Amazon Textract	Resume analysis	<code>AnalyzeDocument</code> (TABLES, FORMS) on the stored S3 object; skills saved to the profile
Amazon SES	Transactional e-mail	<code>SendEmail</code> for confirmations, status updates and manual messages
AWS IAM	Access control	Instance role <code>CloudPMS-EC2-Role</code> (production), IAM user <code>cloudpms-app-user</code> (local development)
MongoDB Atlas	Managed database	Mongoose 8 ODM, indexes for eligibility queries
Hostinger DNS	Domain mapping	A record <code>cloudpms</code> → 65.1.71.208 (TTL 300 s)
Let's Encrypt / Certbot	HTTPS certificate	Certificate for <code>cloudpms.kaifkhan.in</code> , auto-renewal scheduled

4.1 Amazon EC2 (Elastic Compute Cloud)

- **Role:** compute backbone hosting the whole platform (web server, API and static frontend).
- **Instance:** `CloudPMS-Server`, type `t3.micro`, Availability Zone `ap-south-1b`, status checks 3/3 passed.
- **OS and runtime:** Ubuntu Linux; Node.js 22 (v22.23.3) with npm 10.9.9 installed through nvm.
- **Addressing:** Elastic IP `65.1.71.208`; private IP `172.31.10.206`; default VPC.
- **Metadata security:** IMDSv2 is required, protecting the instance credentials against SSRF-style attacks.
- **Firewall (security group `launch-wizard-1`):** HTTP 80 and HTTPS 443 open to the internet; SSH 22 restricted to a single administrator IP (/32) and the AWS-managed prefix list used by EC2 Instance Connect. The API port 5000 is **not** exposed; only Nginx can reach it on localhost.

4.2 Amazon S3 (Simple Storage Service)

S3 is a highly durable object store used for all resume PDFs. The bucket is private; the backend uploads and reads objects through the SDK.

```
s3://cloudpms-resumes-kaif-2026/resumes/<userId>/<timestamp>-<filename>.pdf
```

- **Upload:** Multer `memoryStorage` keeps the PDF in RAM and `PutObjectCommand` streams it to S3, so nothing is written to the EC2 disk. Only PDFs up to 5 MB are accepted.
- **Object metadata:** `ContentType: application/pdf`, `ContentDisposition: inline`, and tags `role=student&userId=<id>`.
- **Replacement:** when a student re-uploads, the previous object is removed with `DeleteObjectCommand`.
- **Secure viewing:** `GET /api/students/resume-view` and `GET /api/placement/students/:studentId/resume-view` return a **presigned URL valid**

for 3,600 seconds (@aws-sdk/s3-request-presigner), so the bucket never has to be public.

4.3 Amazon Textract

- `AnalyzeDocumentCommand` with feature types `TABLES` and `FORMS` is called with the S3 object reference (bucket + key), so Textract reads the file directly from S3.
- Text blocks of type `LINE` are joined and matched against a skill dictionary: javascript, python, java, c++, c#, react, node.js, express, mongodb, sql, mysql, postgresql, aws, docker, kubernetes, html, css, git, typescript, spring boot, django.
- Matched skills are stored in `StudentProfile.skills`. If Textract fails, the error is logged and an empty list is returned, so the resume upload still succeeds (graceful degradation).

4.4 Amazon SES (Simple Email Service)

- `SendEmailCommand` sends HTML e-mail as "CloudPMS Placement Cell" from the SES-verified sender address configured in `AWS_SES_SENDER_EMAIL`.
- E-mail types: `application_confirmation` (when a student applies), `status_update` (when the Placement Cell changes a status) and `manual` (Communication Center).
- Every e-mail, whether sent or failed, is persisted in the MongoDB `EmailLog` collection with recipients, subject, body, related drive/applications, sender and error message.
- At the time of setup the account was in the **SES sandbox** (200 e-mails per 24 hours, 1 e-mail per second): only verified identities can receive mail until production access is granted. The domain `kaifkhan.in` and the sender address are verified in `ap-south-1`.

4.5 AWS IAM (Identity and Access Management)

Two credential models are used with the same code, because the AWS SDK v3 default credential provider chain resolves credentials automatically:

Environment	Identity	Credential source
Local development	IAM user <code>cloudpms-app-user</code> (AmazonS3FullAccess, AmazonSESFullAccess, AmazonTextractFullAccess)	Access key in a git-ignored <code>.env</code> file or <code>~/.aws/credentials</code>
EC2 production	IAM role <code>CloudPMS-EC2-Role</code> attached as an instance profile	Temporary, auto-rotating credentials from the instance metadata service

Aspect	Static key in <code>.env</code>	IAM instance role
Credential rotation	Manual	Automatic
Exposure risk	High (disk, process memory, repository)	None stored on disk
Revocation	Delete key and redeploy	Instant policy change or detach
Audit	IAM Access Analyzer	AWS CloudTrail

Recommended least-privilege policy for the production role (replace the managed full-access policies used during development):

```
{
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Effect": "Allow",
    "Action": ["s3:PutObject", "s3:PutObjectTagging", "s3:GetObject", "s3:DeleteObject"],
    "Resource": "arn:aws:s3:::cloudpms-resumes-kaif-2026/*"
  },
  { "Effect": "Allow", "Action": ["textextract:AnalyzeDocument"], "Resource": "*" },
  { "Effect": "Allow", "Action": ["ses:SendEmail"], "Resource": "*" }
]
}

```

4.6 MongoDB Atlas (Database as a Service)

- Fully managed MongoDB cluster: provisioning, patching, backups and monitoring are handled by Atlas; the EC2 instance only holds the connection string in `MONGO_URI`.
- Access is limited by the Atlas IP access list, which allows the EC2 Elastic IP.
- Mongoose 8 defines schemas, validation and indexes (see Section 7).

4.7 DNS, Nginx, PM2 and HTTPS

- **DNS:** Hostinger A record `cloudpms` → `65.1.71.208` (TTL 300 s).
- **Nginx:** serves the React build from `/var/www/cloudpms/dist` and reverse-proxies `/api/` to Express. `try_files $uri $uri/ /index.html` lets React Router handle deep links and page refreshes.
- **PM2:** runs the API as process `cloudpms`, restarts it on crash, and `pm2 save + pm2 startup` bring it back after an EC2 reboot.
- **HTTPS:** a Let's Encrypt certificate for `cloudpms.kaifkhan.in` was issued with Certbot (expires 2026-12-29) and a scheduled task renews it automatically.

```

server {
    listen 80;
    server_name cloudpms.kaifkhan.in;

    # Frontend: React single-page application
    location / {
        root /var/www/cloudpms/dist;
        try_files $uri $uri/ /index.html;
    }

    # Backend API: reverse proxy to Express (PM2)
    location /api/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

Nginx server block (HTTP listener shown; TLS uses the Certbot certificate in `/etc/letsencrypt/live/cloudpms.kaifkhan.in/`).

5. Technology Stack

Layer	Technology	Details
Frontend framework	React 19 + Vite 8	Single-page application, production bundle served by Nginx
Styling	Tailwind CSS 3	Token-driven design system with light and dark themes; responsive layouts
Routing	React Router DOM 7	Protected routes per role
HTTP client	Axios 1	Interceptor refreshes the access token on 401 and retries the request
Charts / icons	Recharts 3, Lucide React	Admin reports and UI icons
Social login	@react-oauth/google	Google token exchanged at POST /api/auth/google
Backend	Node.js 22 + Express 4	REST API, middleware pipeline
Database	MongoDB Atlas + Mongoose 8	Schemas, validation, compound indexes
Authentication	jsonwebtoken, bcryptjs	Access + refresh JWT; bcrypt with 12 salt rounds
Validation	express-validator 7	Body and parameter validation chains
Uploads	Multer 2 (memoryStorage)	PDF buffer streamed to S3, no disk write
Security middleware	Helmet, CORS, express-rate-limit	Security headers, origin restriction, 20 requests / 15 min on /api/auth in production
Logging	Morgan	combined format in production
AWS SDK	v3: client-s3, client-textract, client-ses, s3-request-presigner	Cloud service integration
Hosting	AWS EC2, Nginx, PM2	Ubuntu server, reverse proxy, process manager

6. Functional Modules & User Roles

6.1 User Roles

Role	Identifier	Key capabilities
Student	student	Manage profile, upload resume, view eligible drives, apply, track application status
Placement Cell	placement_cell	Create/edit/close drives, pipeline management, interview workspace, Communication Center
Administrator	admin	All Placement Cell permissions plus user management, account activation/deactivation, dashboard and reports

6.2 Modules

Authentication module

- E-mail/password registration and login, plus Google OAuth login.
- Short-lived access token with a rotating refresh token in an `httpOnly` cookie; logout invalidates the refresh token in the database.

Student module

- Profile upsert: roll number, branch (MCA, BCA, BBA, BSc IT, BCom), CGPA, backlog count, 10th and 12th percentages.
- Resume upload to S3 with automatic Textract skill extraction.
- Eligible drive list filtered by branch, CGPA, backlogs, deadline and status; one-click apply (one application per drive).
- My applications with live status and remarks.

Placement Cell module

- Drive management: company, role, CTC (LPA), description, eligibility (min CGPA, allowed branches, max backlogs), deadline, drive date, venue.
- Applicant pipeline per drive, filterable by status; update status, remarks and interview date; open an applicant's resume through a presigned URL.
- **Interview Workspace:** timeline of scheduled interviews across active drives with Today / Upcoming / Completed filters and Select / Reject / Reschedule actions.

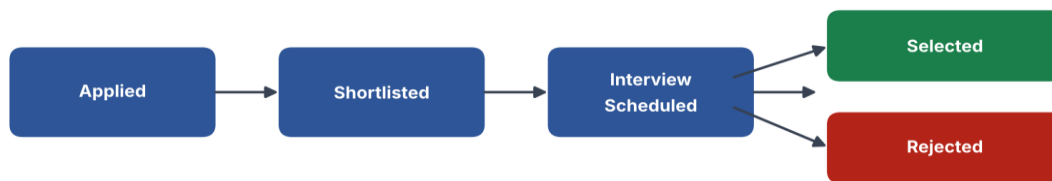
Communication Center

- Compose e-mails to a single student, multiple students, all applicants, shortlisted, interview-scheduled or selected candidates. Recipients are resolved on the server from MongoDB, so arbitrary addresses cannot be injected.
- Full e-mail history with search, type/status filters and detail view, read from the `EmailLog` collection.

Admin module

- Dashboard statistics, paginated user list filterable by role, account enable/disable, and drive-wise application/selection reports with charts.

6.3 Application Status Lifecycle



An email is sent through Amazon SES on every status change

Fig 6.1 — Application status lifecycle.

6.4 Eligibility Rule

A drive is shown to a student only if all three conditions hold (evaluated as one MongoDB query):

```
drive.eligibility.minCgpa      <= student.cgpa
drive.eligibility.maxBacklogs  >= student.backlogCount
student.branch                 IN drive.eligibility.allowedBranches
(and the drive deadline has not passed and the drive is not completed)
```

7. Database Design

MongoDB Atlas hosts five collections modelled with Mongoose. All schemas use automatic `createdAt` / `updatedAt` timestamps.

7.1 User

Field	Type	Constraints
name	String	2–100 characters, required
email	String	Unique, lower-case, valid format
password	String	bcrypt hash, <code>select: false</code> (never returned)
role	String	Enum: student, placement_cell, admin
isVerified	Boolean	Default false
refreshToken	String	<code>select: false</code> ; null means logged out

7.2 StudentProfile (one-to-one with User)

Field	Type	Constraints
user	ObjectId → User	Unique
rollNumber	String	Unique, upper-case
branch	String	Enum: MCA, BCA, BBA, BSc IT, BCom
cgpa	Number	0–10
backlogCount	Number	≥ 0, default 0
resumePath	String	S3 URL; null until uploaded
skills	[String]	Populated by Textract after upload
tenthPercent / twelfthPercent	Number	Optional, 0–100
isPlaced	Boolean	Default false

A compound index on { `branch`, `cgpa`, `backlogCount` } supports the eligibility queries.

7.3 Drive

Field	Type	Constraints
<code>company</code> , <code>jobRole</code>	String	Required
<code>ctc</code>	Number	LPA, ≥ 0
<code>eligibility</code>	Object	<code>minCgpa</code> (0–10), <code>allowedBranches</code> (min 1), <code>maxBacklogs</code> (≥ 0)
<code>deadline</code>	Date	Required
<code>driveDate</code> , <code>venue</code>	Date, String	Optional
<code>postedBy</code>	ObjectId $\rightarrow$ User	Required
<code>status</code>	String	Enum: Upcoming, Ongoing, Completed

7.4 Application

Field	Type	Constraints
<code>student</code>	ObjectId $\rightarrow$ StudentProfile	Required
<code>drive</code>	ObjectId $\rightarrow$ Drive	Required
<code>status</code>	String	Applied, Shortlisted, Interview Scheduled, Selected, Rejected
<code>appliedAt</code>	Date	Default: now
<code>remarks</code>	String	Optional Placement Cell notes

A compound **unique index** on `{ `student`, `drive` }` prevents duplicate applications at the database level.

7.5 EmailLog

Field	Type	Notes
<code>senderEmail</code> , <code>senderName</code>	String	Verified SES sender
<code>recipients</code>	[String]	Resolved on the server
<code>subject</code> , <code>messageText</code>	String	Stored HTML body for reference
<code>type</code>	String	manual, application_confirmation, status_update
<code>status</code>	String	sent or failed
<code>relatedDriveId</code> , <code>relatedApplicationIds</code>	ObjectId refs	Links to Drive / Application
<code>sentBy</code> , <code>errorMessage</code>	ObjectId, String	Triggering user; SES error on failure

8. Authentication, Authorisation & Security

8.1 JWT Strategy: Stateless + Stateful Hybrid

Token	Lifetime	Storage	Transport
Access token	15 minutes	React memory (not localStorage)	Authorization: Bearer header
Refresh token	7 days	MongoDB + httpOnly cookie	Cookie, sent automatically by the browser

- **Stateless part:** the access token carries `userId` and `role`, so protected routes need no database lookup and the API scales horizontally.
- **Stateful part:** the refresh token is stored on the user document; logout sets it to `null`, so a stolen token cannot be reused after logout.
- **Cookie flags:** `httpOnly`; `secure` and `sameSite=none` in production, `lax` in development.

8.2 Role-Based Access Control

Route group	Middleware applied
<code>/api/auth/*</code>	Public, or <code>verifyToken</code> only (me, logout)
<code>/api/students/*</code>	<code>verifyToken + allowRoles('student')</code>
<code>/api/placement/*</code> , <code>/api/placement-cell/emails/*</code>	<code>verifyToken + allowRoles('placement_cell', 'admin')</code>
<code>/api/admin/*</code>	<code>verifyToken + allowRoles('admin')</code>

8.3 Security Measures

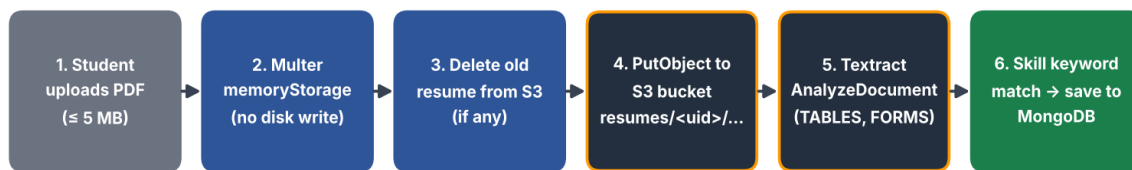
Measure	Implementation
Password storage	bcryptjs, 12 salt rounds; minimum 8 characters, one upper-case letter and one digit
Security headers	Helmet on all responses
CORS	Restricted to <code>CLIENT_URL</code> , credentials enabled for the refresh cookie
Rate limiting	20 requests per 15 minutes on <code>/api/auth</code> in production
Input validation	express-validator on user-facing routes; JSON body limited to 10 KB
Secrets	<code>.env</code> excluded from Git; <code>.env.example</code> provided as a template
AWS credentials	IAM instance role on EC2; no keys on the server
S3 access	Private bucket; time-limited presigned URLs (3,600 s)
Transport security	HTTPS with Let's Encrypt certificate
Network	Security group exposes only 80/443 publicly; SSH limited to a trusted IP; API port bound behind Nginx
Instance metadata	IMDSv2 required

9. API Reference & Processing Flows

9.1 REST API Summary

Method	Endpoint	Role	Description
POST	/api/auth/register	Public	Register with name, e-mail, password, role
POST	/api/auth/login	Public	E-mail and password login
POST	/api/auth/google	Public	Google OAuth token exchange
POST	/api/auth/refresh	Cookie	Rotate access and refresh tokens
GET / POST	/api/auth/me, /logout	Bearer	Current user; invalidate refresh token
POST / PUT / GET	/api/students/profile	Student	Create, update, read academic profile
POST	/api/students/resume	Student	Upload PDF to S3 and extract skills
GET	/api/students/resume-view	Student	Presigned URL for own resume
GET	/api/students/drives	Student	Drives the student is eligible for
POST	/api/students/drives/:driveId/apply	Student	Apply to a drive
GET	/api/students/applications	Student	Own applications and status
POST / GET / PUT	/api/placement/drives	Placement, Admin	Create, list, edit drives
PATCH	/api/placement/drives/:id/close	Placement, Admin	Close a drive
GET	/api/placement/drives/:id/applicants	Placement, Admin	List applicants (<i>?status=</i>)
PATCH	/api/placement/applications/:applId/status	Placement, Admin	Update status, remarks, interview date; sends e-mail
GET	/api/placement/students/:studentId/resume-view	Placement, Admin	Presigned URL for a student's resume
POST / GET	/api/placement-cell/emails/send, /	Placement, Admin	Send manual e-mail; e-mail history
GET	/api/admin/dashboard, /users, /reports/drives	Admin	Statistics, users, drive reports
PATCH	/api/admin/users/:id/toggle	Admin	Activate or deactivate an account
GET	/api/health	Public	API status and AWS configuration

9.2 Resume Upload and Processing Flow



Textract failure is non-fatal: upload still succeeds and an empty skill list is returned

Fig 9.1 — Resume upload and processing flow.

9.3 SES Notification Flow

1. A student applies (or the Placement Cell updates an application status or sends a manual message).
2. The application record is saved in MongoDB first.
3. `sendEmail(to, subject, htmlBody, logData)` calls `SendEmailCommand` through the SES client.
4. The result (sent or failed, with the error text) is written to `EmailLog`.
5. A failure is caught and logged: the status change is never rolled back because of an e-mail problem.

10. Environment Configuration Reference (.env)

Variable	Required	Description
<code>NODE_ENV</code>	Yes	<code>development</code> or <code>production</code>
<code>PORT</code>	Optional	Express port (5000)
<code>CLIENT_URL</code>	Yes	Allowed CORS origin (frontend URL)
<code>MONGO_URI</code>	Yes	MongoDB Atlas connection string
<code>JWT_ACCESS_SECRET</code> / <code>JWT_REFRESH_SECRET</code>	Yes	Signing secrets (64-byte random hex)
<code>JWT_ACCESS_EXPIRES_IN</code> / <code>JWT_REFRESH_EXPIRES_IN</code>	Optional	Defaults 15m / 7d
<code>AWS_REGION</code>	Yes	<code>ap-south-1</code>
<code>AWS_S3_BUCKET_NAME</code>	Yes	<code>cloudpms-resumes-kaif-2026</code>
<code>AWS_SES_SENDER_EMAIL</code>	Yes	SES-verified sender address
<code>BCRYPT_SALT_ROUNDS</code>	Optional	Default 12
<code>GOOGLE_CLIENT_ID</code>	Optional	Google OAuth client ID
<code>AWS_ACCESS_KEY_ID</code> / <code>AWS_SECRET_ACCESS_KEY</code>	Local only	IAM user keys for development; not set on EC2 because the instance role is used

The client uses `VITE_API_BASE_URL=/api` (Nginx proxies `/api` to Express) and optionally `VITE_GOOGLE_CLIENT_ID`. Secrets are never committed to GitHub.

11. Production Deployment & Setup — Screenshots

This section documents the end-to-end cloud setup of CloudPMS, from resource provisioning to the live application, captured during the deployment.

11.1 Amazon S3 Bucket

A private S3 bucket `cloudpms-resumes-kaif-2026` was created in `ap-south-1` to store student resumes under a `resumes/<userId>/` prefix. The screenshots show the bucket after student uploads, the per-user folders, the object properties, and a resume opened through a presigned URL.

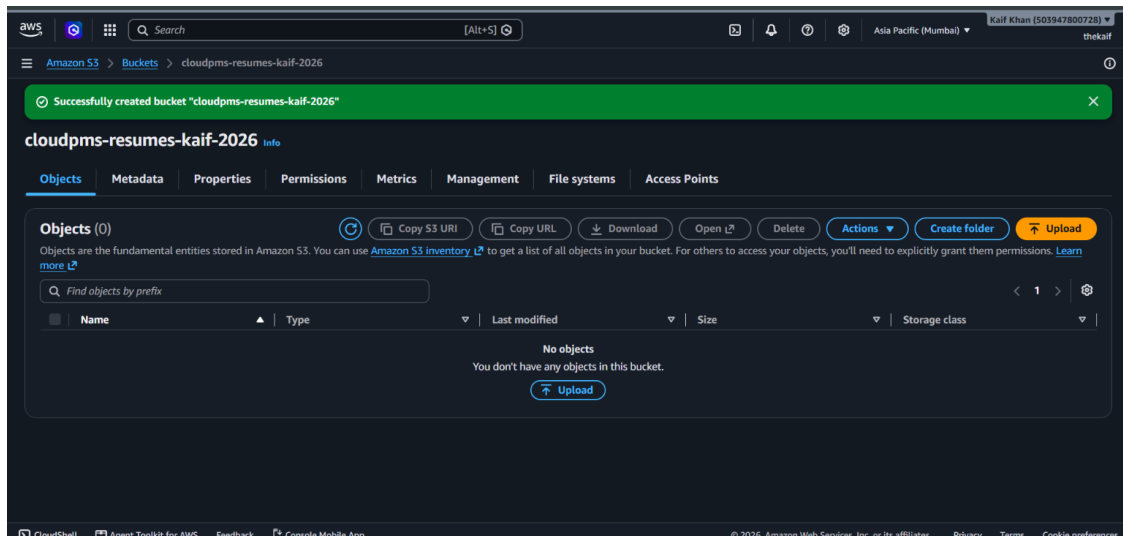


Fig 11.1a — S3 bucket "cloudpms-resumes-kaif-2026" created successfully in Asia Pacific (Mumbai).

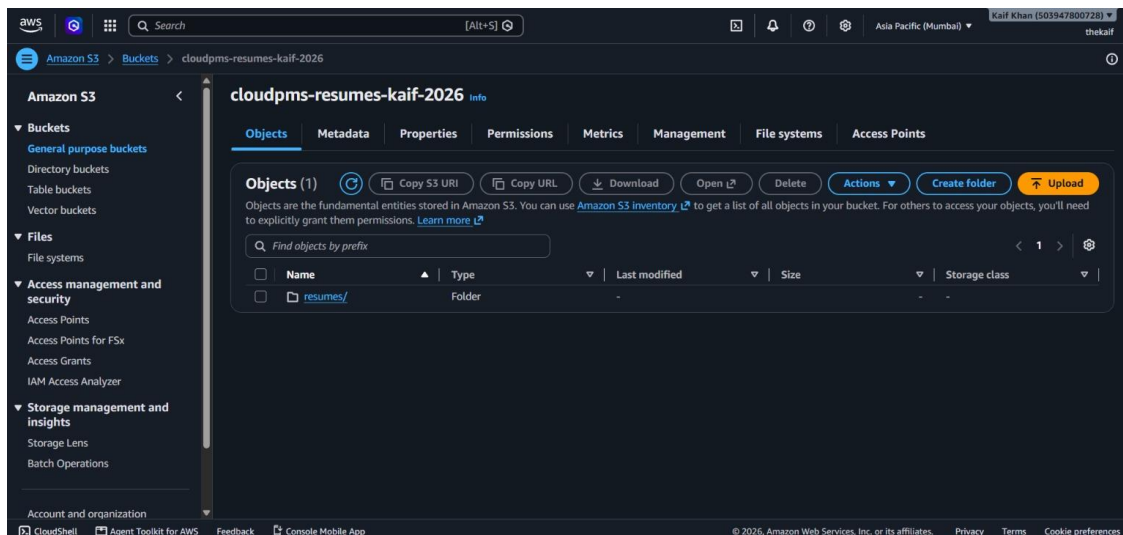


Fig 11.1b — Bucket contents: the resumes/ prefix.

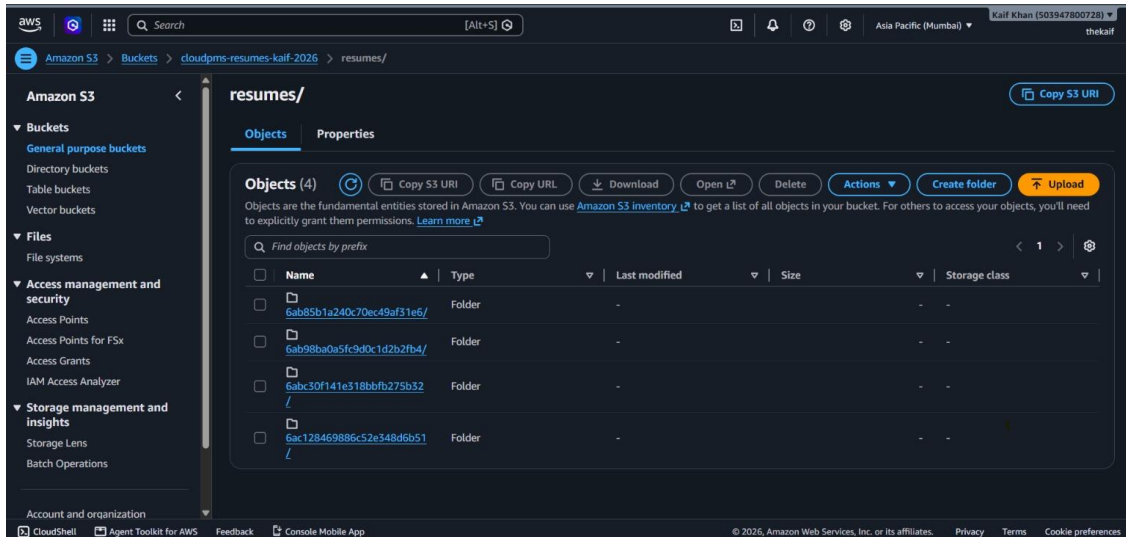


Fig 11.1c — One folder per student (user ID) under resumes/.

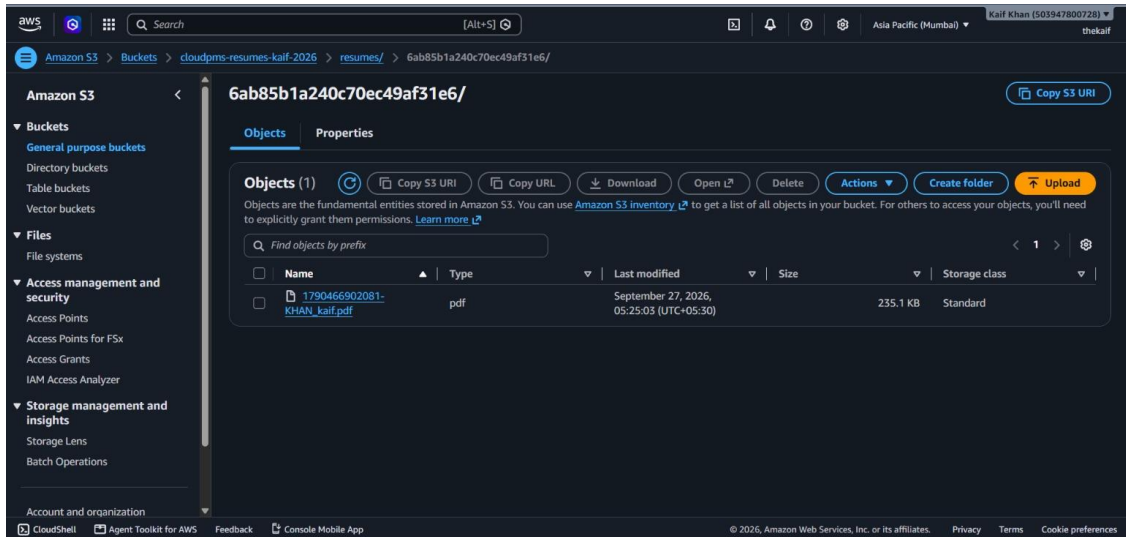


Fig 11.1d — A student folder holding the uploaded PDF (timestamp + file name).

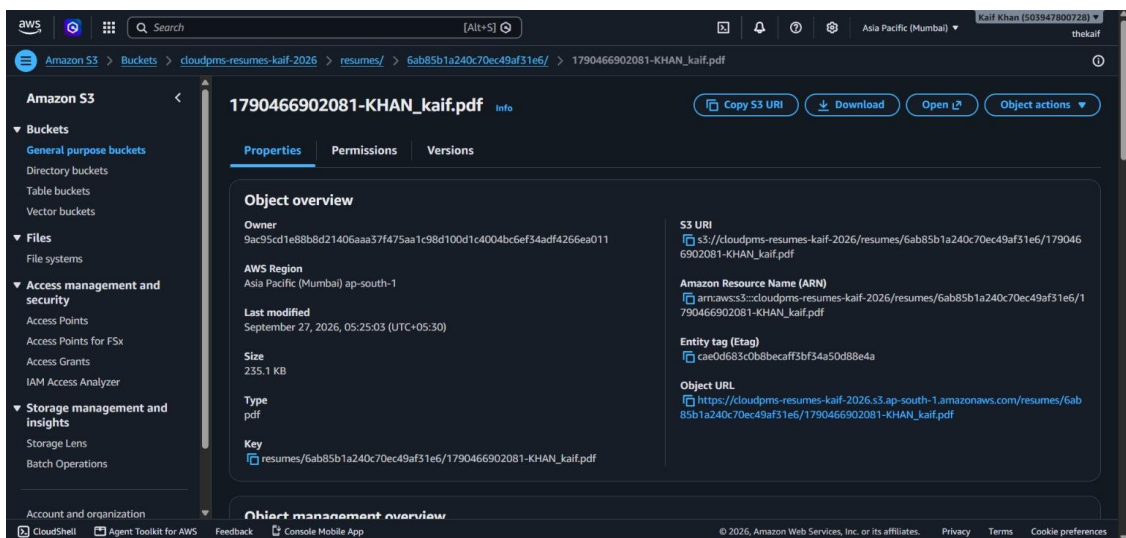


Fig 11.1e — Object properties: S3 URI, ARN, key, size and Mumbai region.

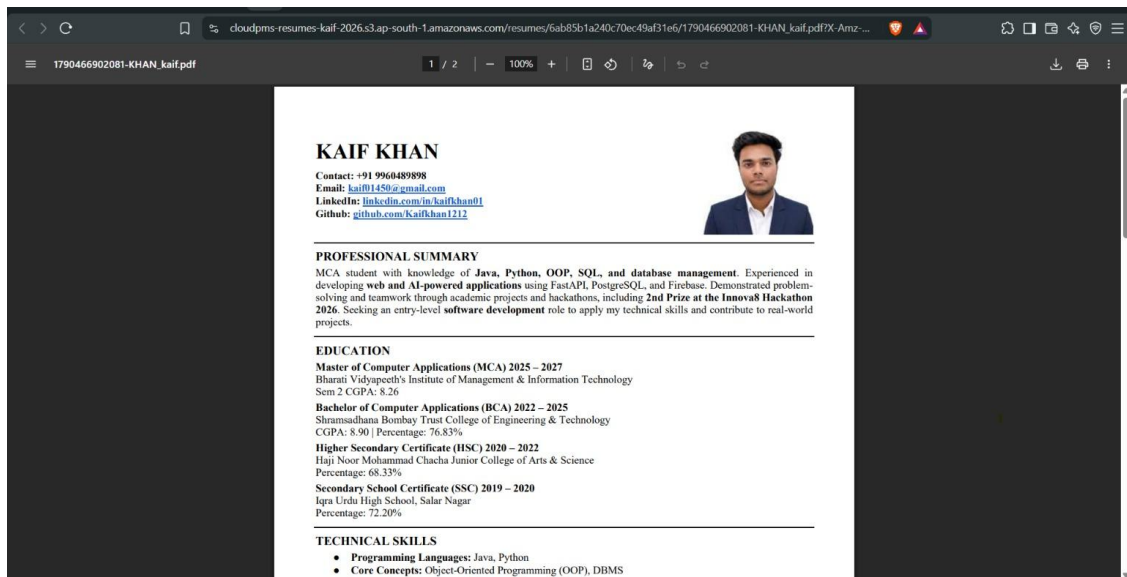


Fig 11.1f — Resume opened through a time-limited presigned URL (X-Amz query parameters).

11.2 IAM User and Access

For local development an IAM user `cloudpms-app-user` was created with S3, SES and Textract permissions and an access key for application code. In production, the EC2 instance role replaces this key (see 11.5).

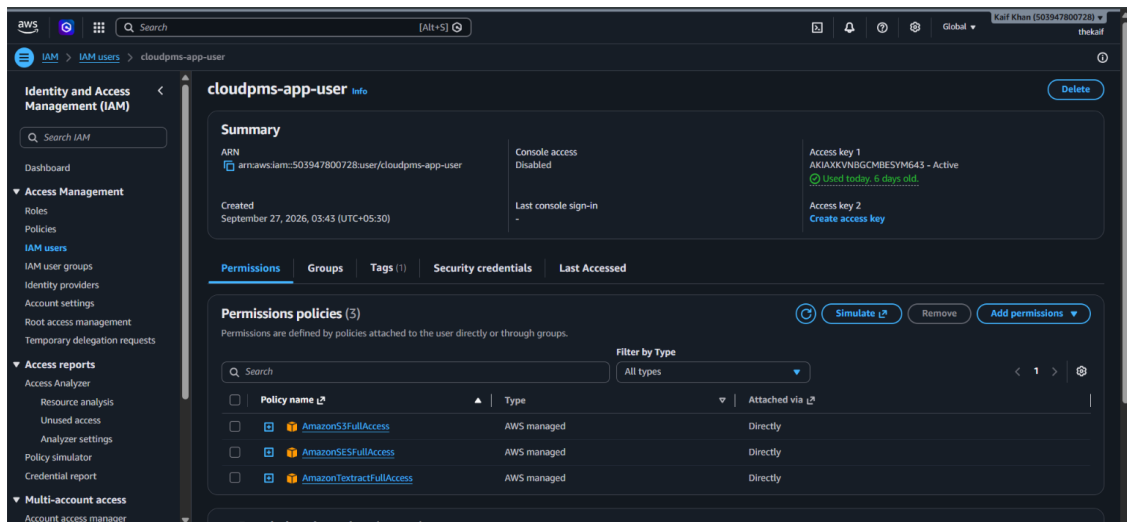


Fig 11.2a — IAM user `cloudpms-app-user` and its attached permission policies.

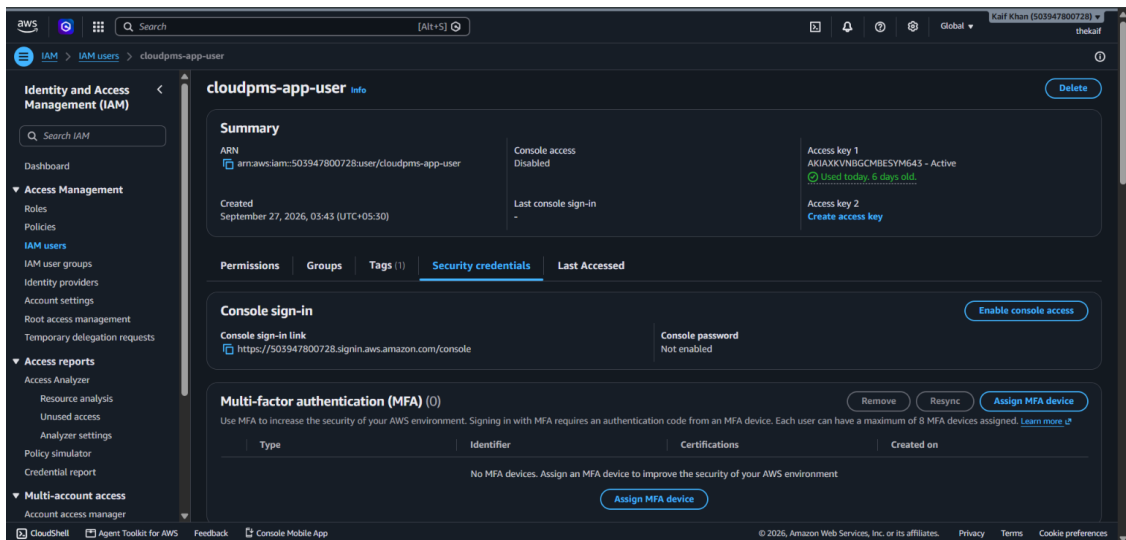


Fig 11.2b — Security credentials tab: console access disabled for the application user.

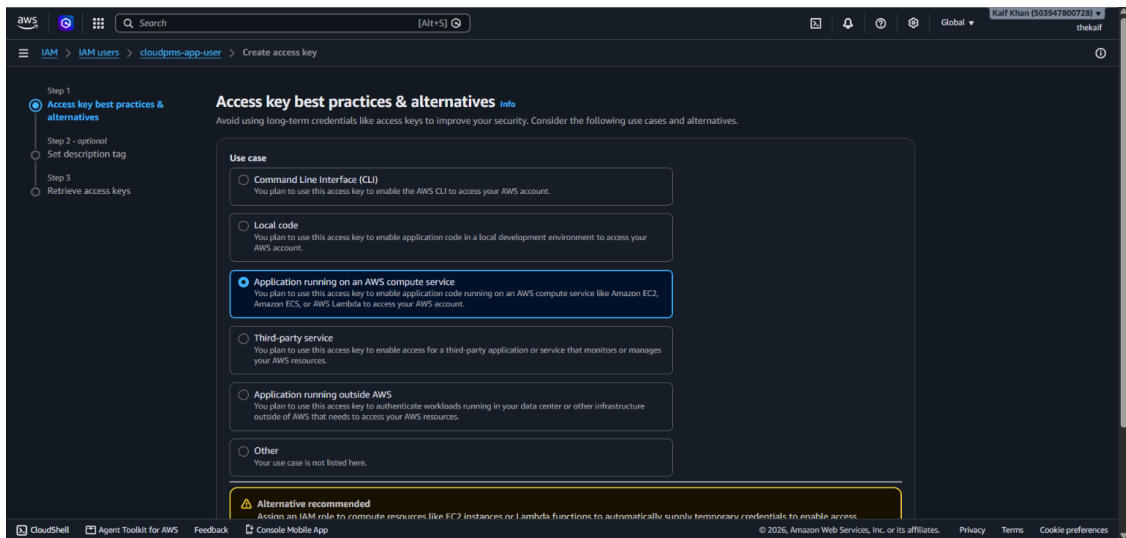


Fig 11.2c — Access key use case "Application running on an AWS compute service".

11.3 Amazon SES Setup & Identity Verification

Amazon SES was configured in `ap-south-1` to send application confirmations and status e-mails. A sender e-mail identity was verified, together with the domain `kaiifkhan.in`.

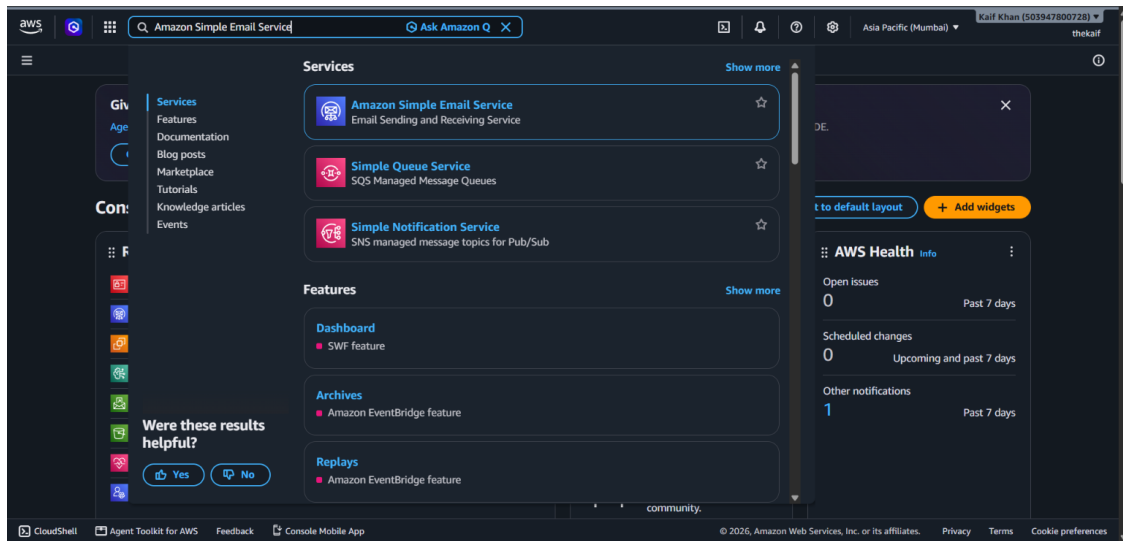


Fig 11.3a — Searching for Amazon Simple Email Service in the AWS console.

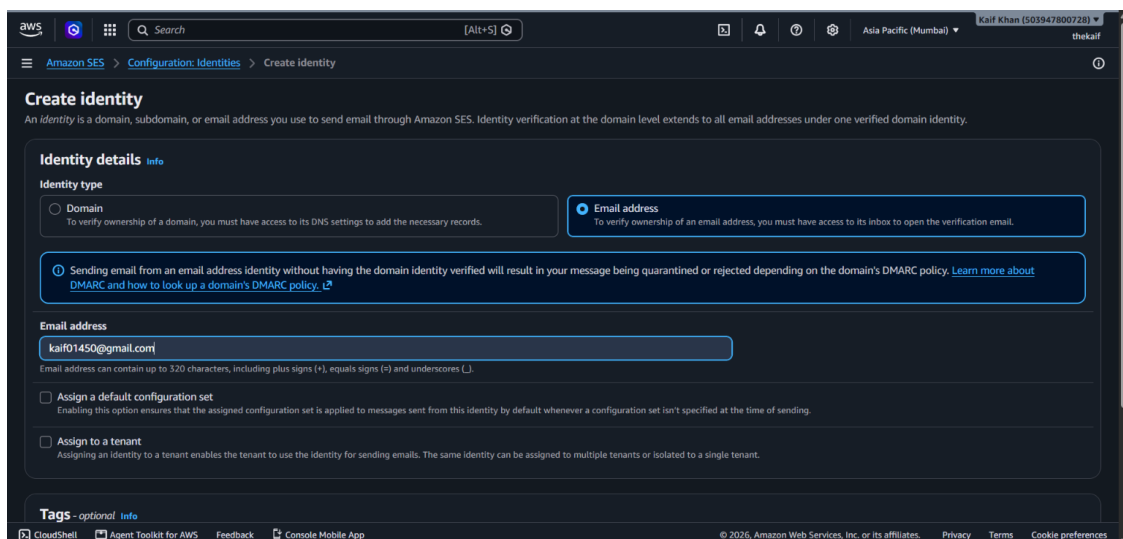


Fig 11.3b — Creating a new e-mail address identity.

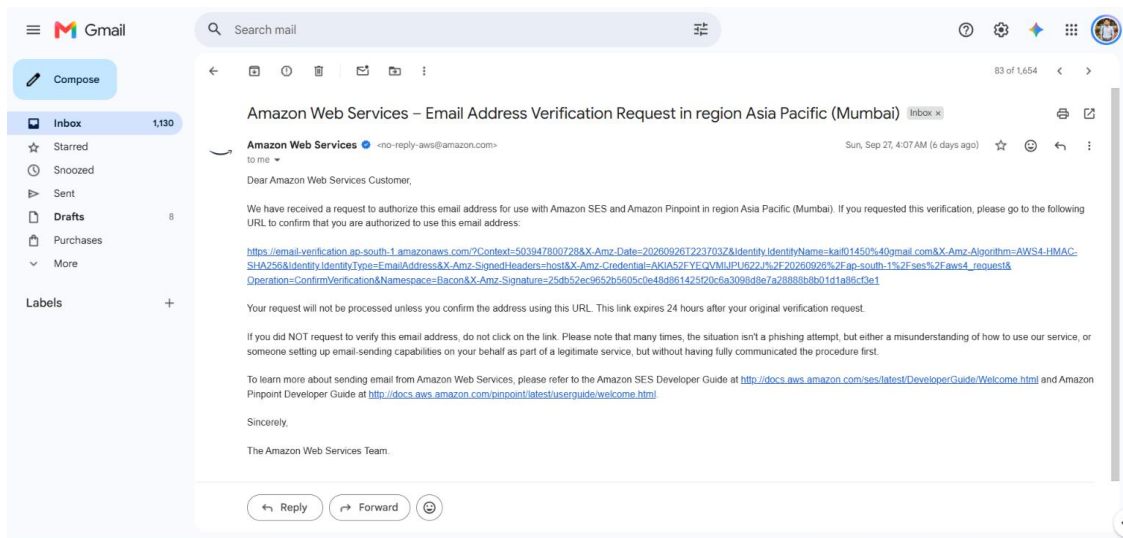


Fig 11.3c — AWS verification request received in the inbox.

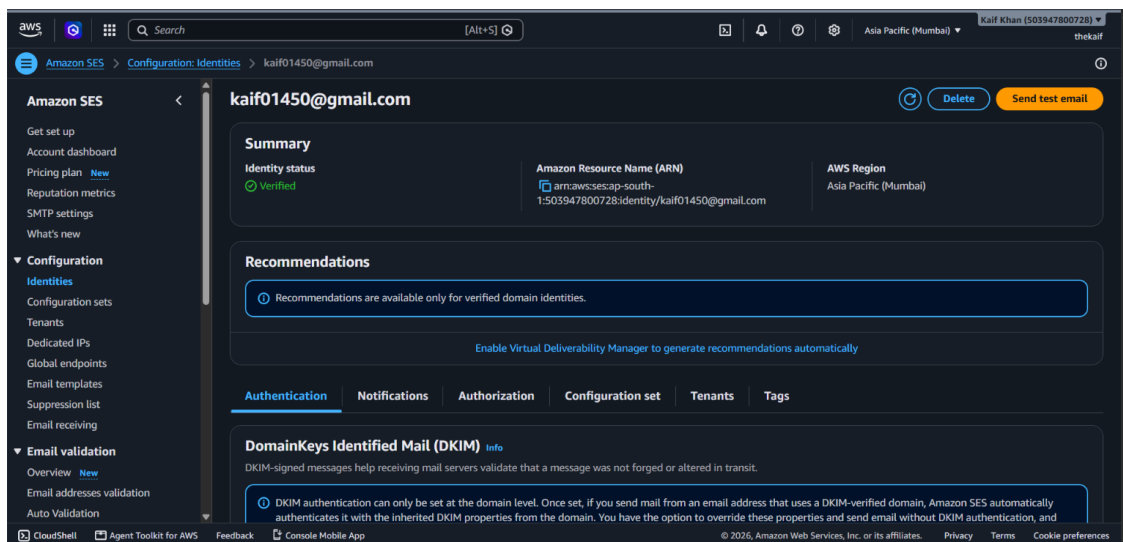


Fig 11.3d — Verified identity status in the SES console.

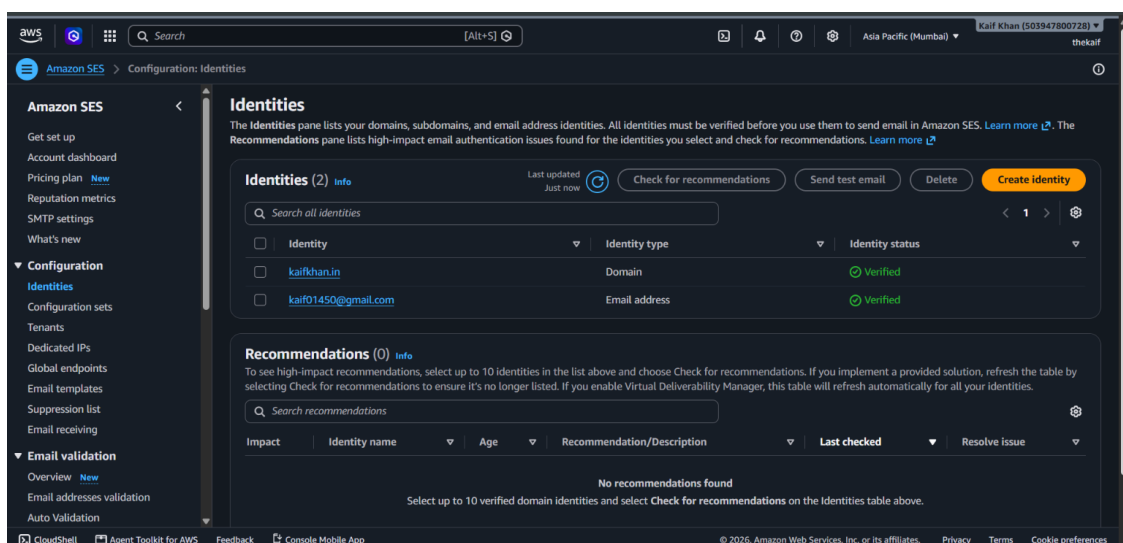


Fig 11.3e — SES identities list: domain kaifkhan.in and the sender address are both verified.

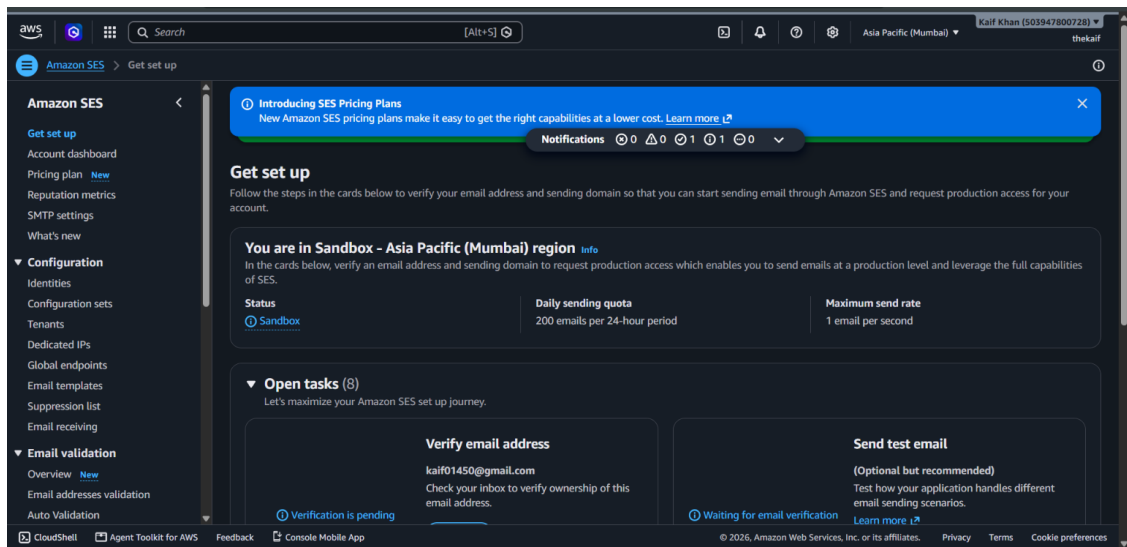


Fig 11.3f — SES account in sandbox mode: 200 e-mails per 24 hours, 1 e-mail per second.

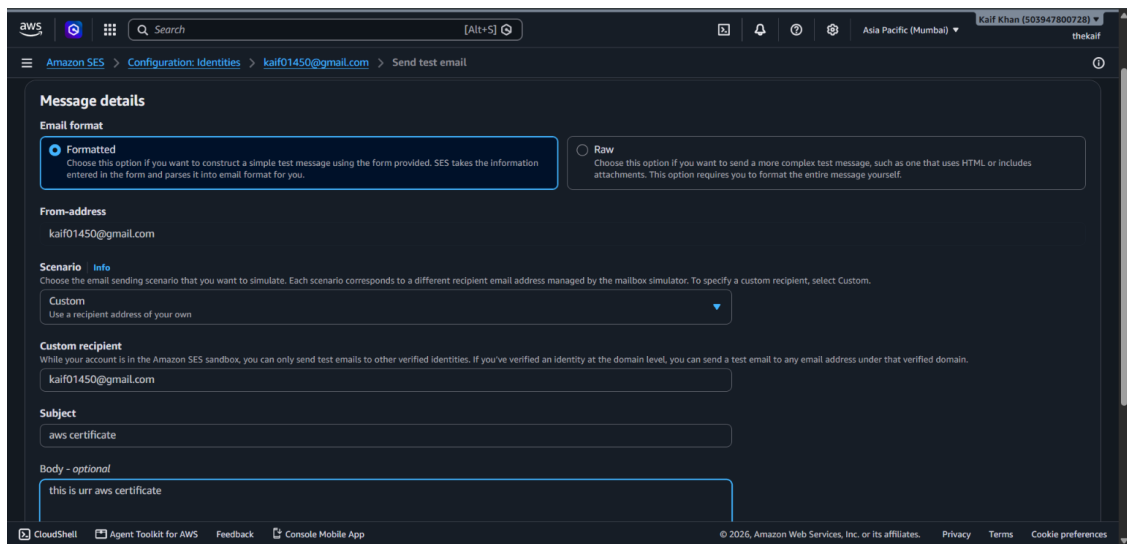


Fig 11.3g — Sending a test e-mail from the SES console.



Fig 11.3h — Test e-mail received in the inbox, confirming SES delivery.

11.4 Amazon EC2 Instance

An Ubuntu `t3.micro` instance named `CloudPMS-Server` was launched in `ap-south-1b`, a security group was created, an Elastic IP was associated and the IAM role `CloudPMS-EC2-Role` was attached.

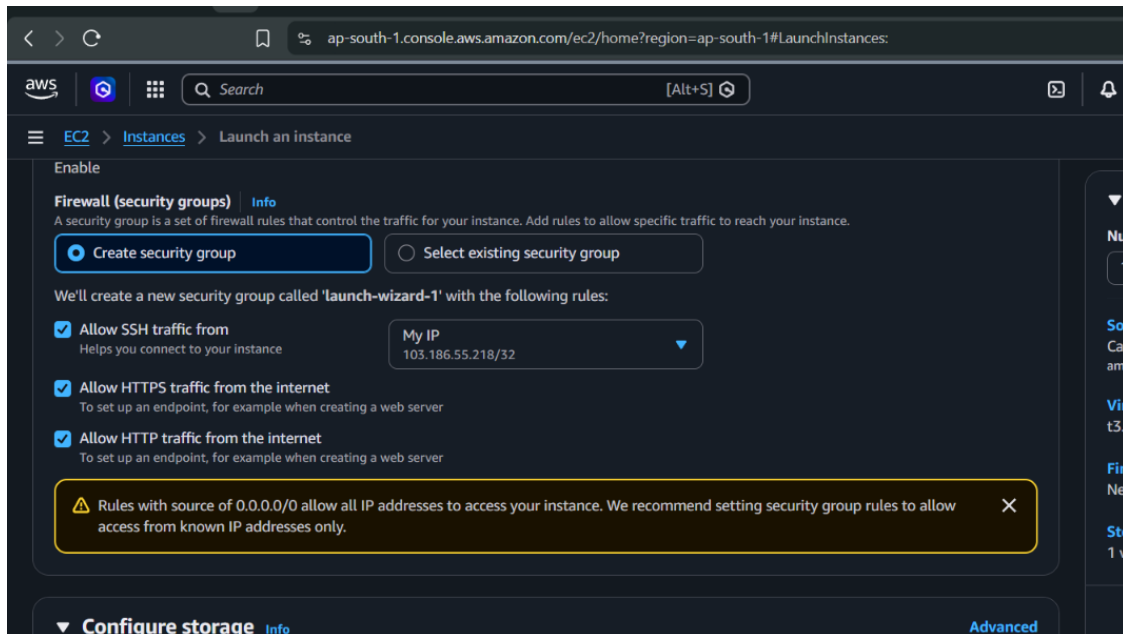


Fig 11.4a — Launch wizard: new security group allowing SSH (from my IP), HTTPS and HTTP.

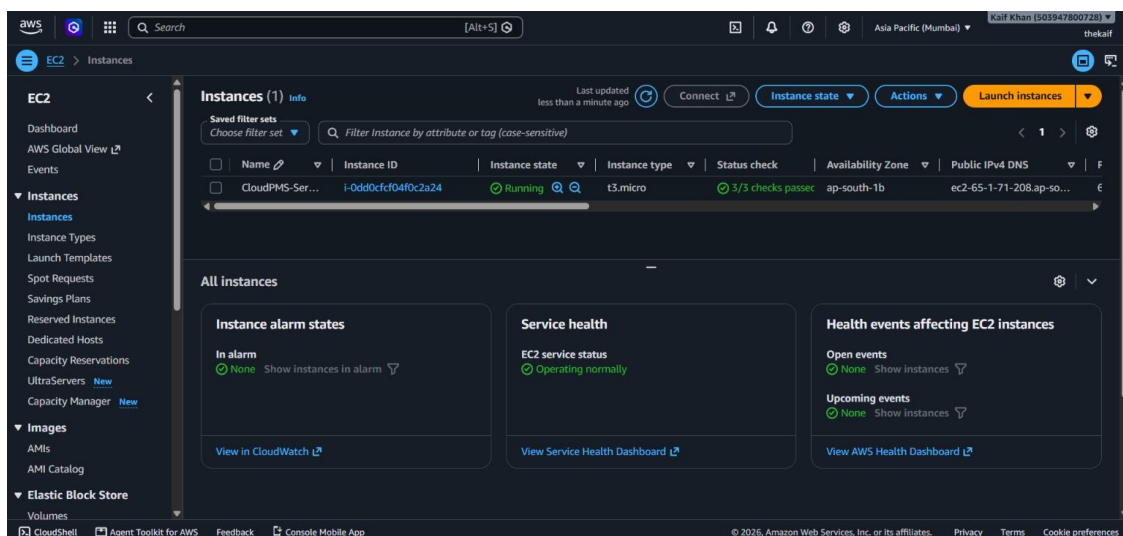


Fig 11.4b — Instance running with 3/3 status checks passed.

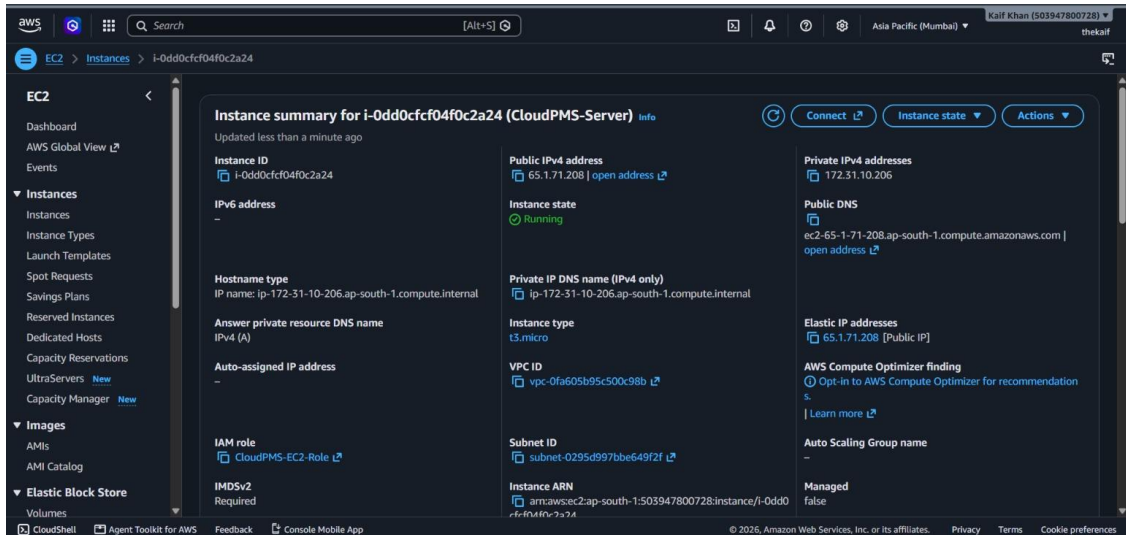


Fig 11.4c — Instance summary: Elastic IP 65.1.71.208, IAM role CloudPMS-EC2-Role, IMDSv2 required.

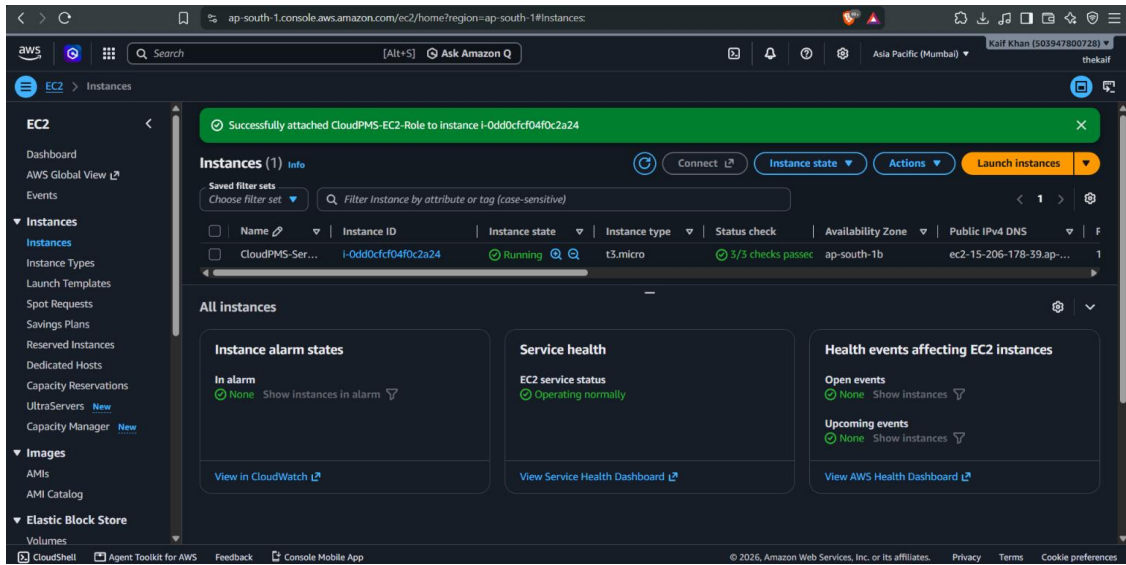


Fig 11.4d — IAM role CloudPMS-EC2-Role successfully attached to the instance.

11.5 Security Group Configuration

Inbound rules allow HTTP (80) and HTTPS (443) from anywhere and SSH (22) only from a trusted administrator address and the EC2 Instance Connect prefix list. The Node.js port 5000 is intentionally closed to the internet; Nginx reaches it on localhost.

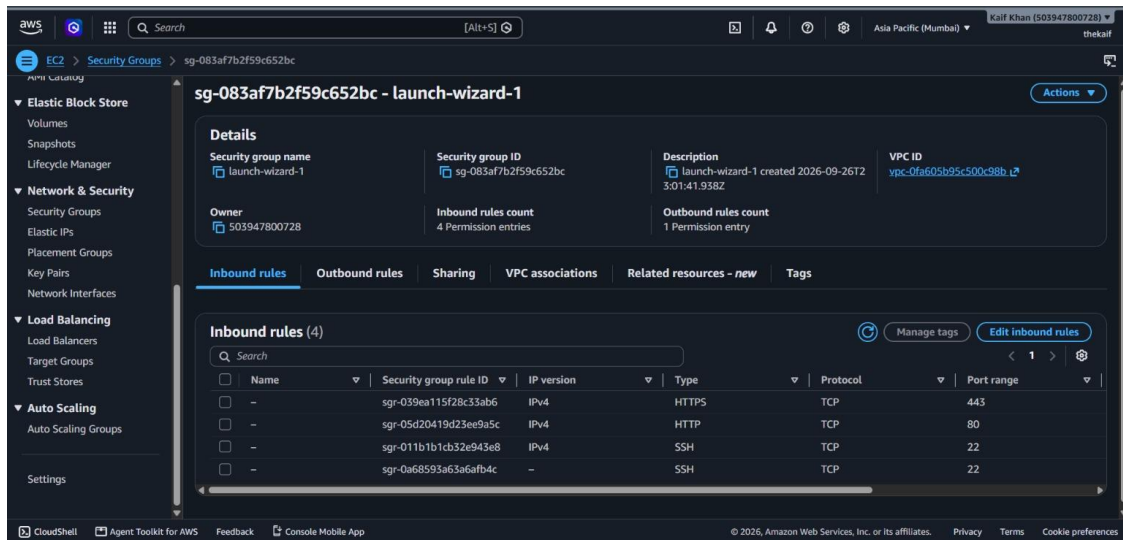


Fig 11.5a — Security group launch-wizard-1 with four inbound rules.

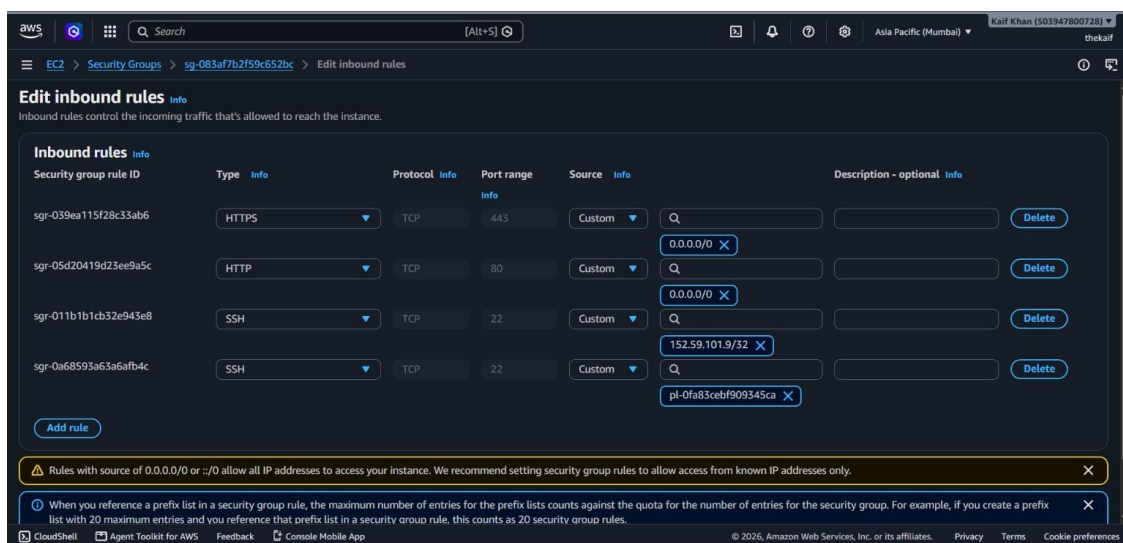


Fig 11.5b — Edit inbound rules: HTTPS, HTTP and restricted SSH sources.

11.6 Connecting to the Server

The instance was reached over SSH with the key pair, and also through EC2 Instance Connect in the browser.

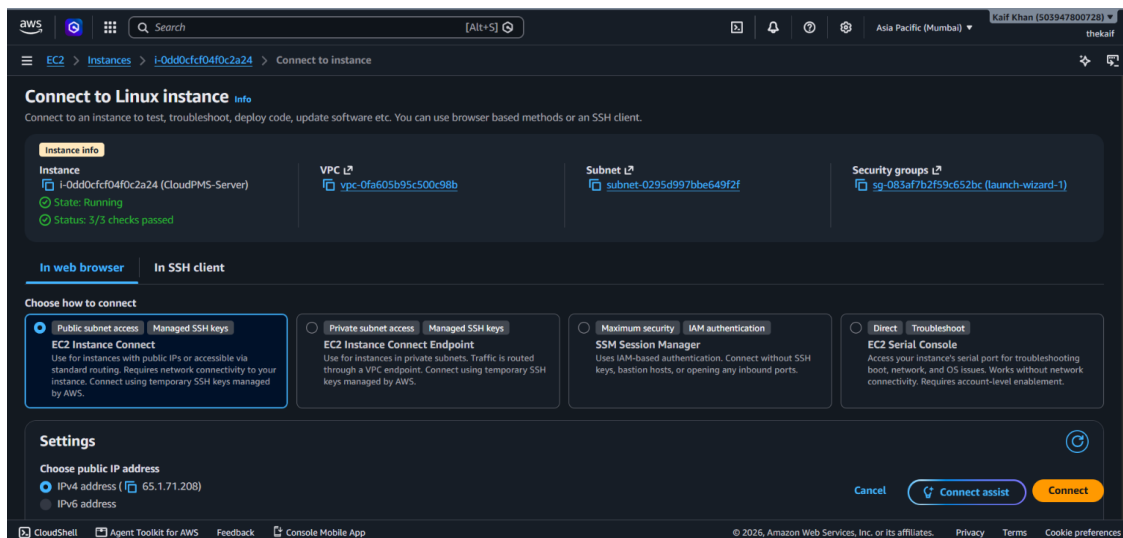


Fig 11.6a — Connect to Linux instance options (EC2 Instance Connect).

```
PS D:\Downloads> ssh -i .\cloudpms-server-key.pem ubuntu@65.1.71.208
The authenticity of host '65.1.71.208 (65.1.71.208)' can't be established.
ED25519 key fingerprint is SHA256:0r/eKwJSH+AoakHCVr95RwcXAvdpYcG0007XJBwdeI0.
This host key is known by the following other names/addresses:
C:\Users\Lenovo\.ssh\known_hosts:1: 15.206.178.39
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '65.1.71.208' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 29 18:08:44 UTC 2026

System load: 0.08      Temperature:   -273.1 C
Usage of /:  42.2% of 6.61GB   Processes:    118
Memory usage: 34%      Users logged in: 0
Swap usage:  0%          IPv4 address for ens5: 172.31.10.206

 * Ubuntu Pro delivers the most comprehensive open source security and
compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

7 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Sat Sep 26 23:20:44 2026 from 103.186.55.218
```

Fig 11.6b — SSH session established with the private key (ubuntu@65.1.71.208).

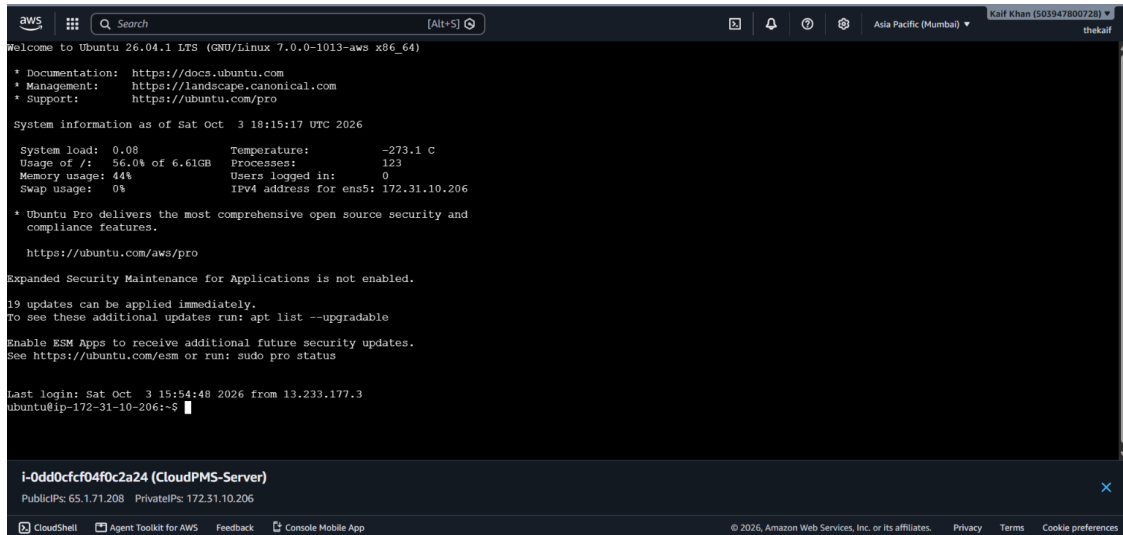


Fig 11.6c — Browser-based terminal session on Ubuntu.

11.7 Server Preparation: Packages and Node.js

The operating system packages were refreshed, Git and Nginx were installed, and Node.js 22 was installed through nvm.

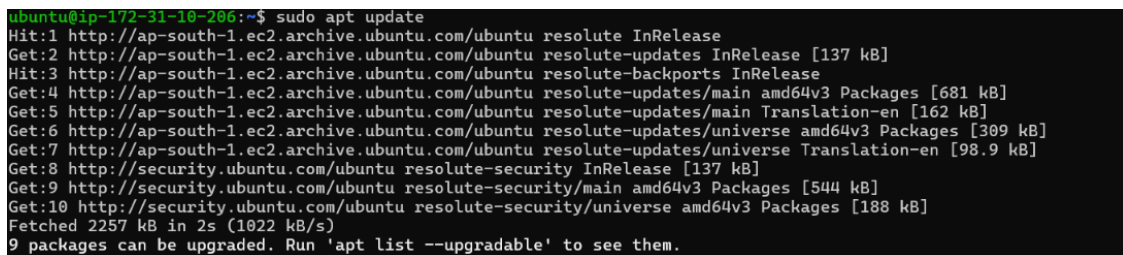


Fig 11.7a — `sudo apt update` refreshing the package lists.

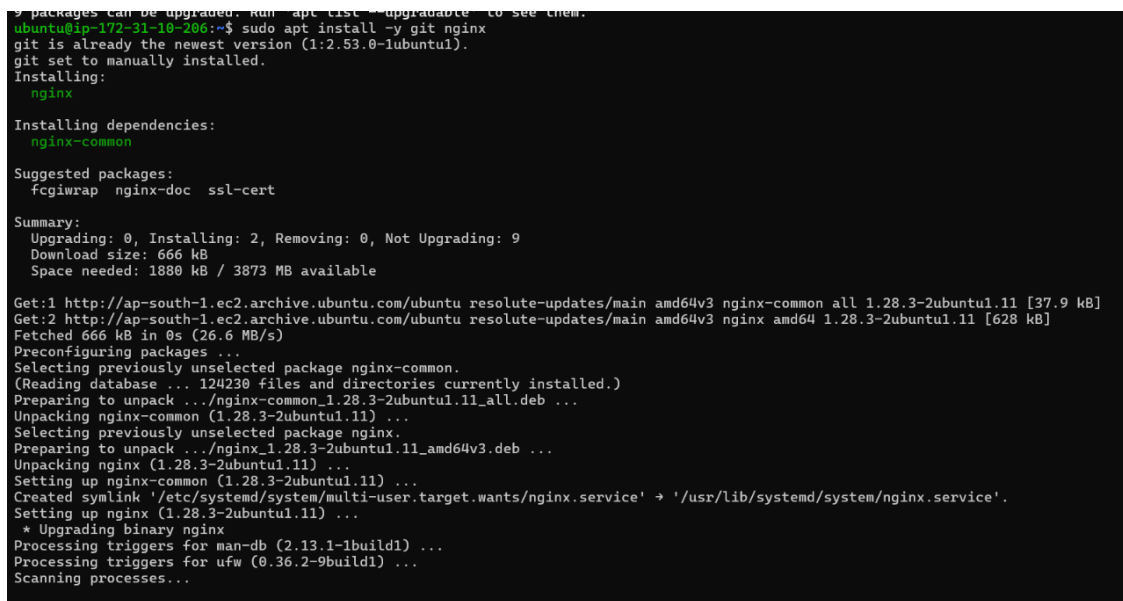


Fig 11.7b — Installing Git and Nginx with apt.

```

ubuntu@ip-172-31-10-206:~$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total   Spent    Left     Speed
100 17883 100 17883    0     0  1.14M    0     0      0
=> Downloading nvm from git to '/home/ubuntu/.nvm'
=> Cloning into '/home/ubuntu/.nvm'...
remote: Enumerating objects: 481, done.
remote: Counting objects: 100% (481/481), done.
remote: Compressing objects: 100% (404/404), done.
remote: Total 481 (delta 68), reused 245 (delta 51), pack-reused 0 (from 0)
Receiving objects: 100% (481/481), 506.16 KiB | 23.01 MiB/s, done.
Resolving deltas: 100% (68/68), done.
* (HEAD detached at FETCH_HEAD)
master
=> Compressing and cleaning up git repository

=> Appending nvm source string to /home/ubuntu/.bashrc
=> Appending bash_completion source string to /home/ubuntu/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
ubuntu@ip-172-31-10-206:~$

```

Fig 11.7c — Installing nvm (Node Version Manager).

```

ubuntu@ip-172-31-10-206:~$ nvm install 22
Downloading and installing node v22.23.3...
Downloading https://nodejs.org/dist/v22.23.3/node-v22.23.3-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v22.23.3 (npm v10.9.9)
Creating default alias: default -> 22 (-> v22.23.3 *)

```

Fig 11.7d — Installing Node.js 22 with nvm.

```

ubuntu@ip-172-31-10-206:~$
Resolving deltas: 100% (68/68), done.
* (HEAD detached at FETCH_HEAD)
master
=> Compressing and cleaning up git repository

=> Appending nvm source string to /home/ubuntu/.bashrc
=> Appending bash_completion source string to /home/ubuntu/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
ubuntu@ip-172-31-10-206:~$ source ~/.bashrc
ubuntu@ip-172-31-10-206:~$ command -v nvm
nvm
ubuntu@ip-172-31-10-206:~$ node -v
Command 'node' not found, but can be installed with:
sudo apt install nodejs
ubuntu@ip-172-31-10-206:~$ node --v
Command 'node' not found, but can be installed with:
sudo apt install nodejs
ubuntu@ip-172-31-10-206:~$ node --version
Command 'node' not found, but can be installed with:
sudo apt install nodejs
ubuntu@ip-172-31-10-206:~$ nvm install 22
Downloading and installing node v22.23.3...
Downloading https://nodejs.org/dist/v22.23.3/node-v22.23.3-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v22.23.3 (npm v10.9.9)
Creating default alias: default -> 22 (-> v22.23.3 *)
ubuntu@ip-172-31-10-206:~$ nvm alias default 22
default -> 22 (-> v22.23.3)
ubuntu@ip-172-31-10-206:~$ nvm use 22
Now using node v22.23.3 (npm v10.9.9)
ubuntu@ip-172-31-10-206:~$ node -v
v22.23.3
ubuntu@ip-172-31-10-206:~$ npm -v
10.9.9
ubuntu@ip-172-31-10-206:~$

```

Fig 11.7e — Verifying node -v (v22.23.3) and npm -v (10.9.9).

11.8 Cloning the Repository & Environment Configuration

The repository was cloned from GitHub, the exact environment variable names used by the code were listed, and `server/.env` was created on the server.

```

ubuntu@ip-172-31-10-206:~$ git clone https://github.com/Kaifkhan1212/CloudPMS-PlacementPortal.git

```

Fig 11.8a — Cloning CloudPMS-PlacementPortal from GitHub.

```
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ grep -Rho "process\.env\.[A-Za-z0-9_]*" server --exclude-dir=node_modules | sort -u
process.env.AWS_REGION
process.env.AWS_S3_BUCKET_NAME
process.env.AWS_SES_SENDER_EMAIL
process.env.BCRYPT_SALT_ROUNDS
process.env.CLIENT_URL
process.env.GOOGLE_CLIENT_ID
process.env.JWT_ACCESS_EXPIRES_IN
process.env.JWT_ACCESS_SECRET
process.env.JWT_REFRESH_EXPIRES_IN
process.env.JWT_REFRESH_SECRET
process.env.MONGO_URI
process.env.NODE_ENV
process.env.PORT
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ |
```

Fig 11.8b — Listing the environment variables referenced by the server code.

```
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ nano server/.env
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ ls -la server/.env
-rw-rw-r-- 1 ubuntu ubuntu 1602 Sep 29 18:42 server/.env
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ grep -E '^(NODE_ENV|PORT|CLIENT_URL|AWS_REGION|AWS_S3_BUCKET_NAME|AWS_SES_SENDER_EMAIL|BCRYPT_SALT_ROUND
S)' server/.env
NODE_ENV=development
PORT=5000
CLIENT_URL=http://65.1.71.208
AWS_REGION=ap-south-1
AWS_S3_BUCKET_NAME=cloudpms-resumes-kaif-2026
AWS_SES_SENDER_EMAIL=kaif01450@gmail.com
BCRYPT_SALT_ROUNDS=12
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ git status --short
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ nano server/.env
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ |
```

Fig 11.8c — Creating and checking server/.env (non-secret values shown).

```
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/server$ ls -la .env
-rw-rw-r-- 1 ubuntu ubuntu 1602 Sep 29 18:42 .env
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/server$ |
```

Fig 11.8d — The .env file exists with restricted permissions.

```
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ cd ~/CloudPMS-PlacementPortal/server
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/server$ npm ci
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead
added 194 packages, and audited 195 packages in 6s

31 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 10.9.9 -> 12.1.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.1.0
npm notice To update run: npm install -g npm@12.1.0
npm notice
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/server$ |
```

Fig 11.8e — npm ci installing 194 backend packages with 0 vulnerabilities.

11.9 Starting the Backend & Health Check

The API was started once manually to confirm the MongoDB Atlas connection and the AWS configuration before handing it over to PM2.

```
BCRYPT_SALT_ROUNDS=12
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/server$ npm start

> cloudpms-server@1.0.0 start
> node server.js

✅ MongoDB connected: ac-xpapnqw-shard-00-01.okqybkg.mongodb.net

🚀 CloudPMS Server running in [development] mode
  → Local: http://localhost:5000
  → Health: http://localhost:5000/api/health
```

Fig 11.9a — npm start: MongoDB connected and CloudPMS server running on port 5000.

```
Last login: Tue Sep 29 18:10:45 2026 from 103.186.55.218
ubuntu@ip-172-31-10-206:~$ curl http://127.0.0.1:5000/api/health
{"statusCode":200,"data":{"status":"ok","environment":"development","timestamp":"2026-09-29T18:51:59.122Z","aws":{"configured":true,"region":"ap-south-1"},"s3Bucket":{"cloudpms-resumes-kaif-2026"},"sesSender":"kaif0145@gmail.com"},"textReady":true},"message":"CloudPMS API is healthy","success":true}ubuntu@ip-172-31-10-206:~$
```

Fig 11.9b — curl /api/health returning status ok, region ap-south-1, S3 bucket, SES sender and textractReady: true.

11.10 PM2 Process Management

PM2 was installed globally to keep the API running in the background with automatic restart on crash and on reboot.

[illegible]

Fig 11.10a — Installing PM2 globally with npm.

```

ubuntu@ip-172-31-10-206:~$ sudo pm2 uninstall systemd
ubuntu@ip-172-31-10-206:~$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
ubuntu@ip-172-31-10-206:~$ pm2 status

```

id	name	namespace	version	mode	pid	uptime	U	status	cpu	mem	user	watching
0	cloudpms	default	1.0.0	fork	3899	4m	0	online	0%	103.5mb	ubuntu	disabled

```

host metrics | cpu: 0.1% | ram usage: 48.5% | ens5: 0.001mb/s 0mb/s | disk: 0mb/s 0.006mb/s
ubuntu@ip-172-31-10-206:~$ cd ~/cloudpms-PlacementPortal/client

```

Fig 11.10b — pm2 save and pm2 status: process cloudpms online.

11.11 Building the React Frontend

```
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal$ cd ~/CloudPMS-PlacementPortal/client
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ pwd
/home/ubuntu/CloudPMS-PlacementPortal/client
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ ls -la
total 160
drwxrwxr-x 4 ubuntu ubuntu 4096 Sep 29 18:29 .
drwxrwxr-x 6 ubuntu ubuntu 4096 Sep 29 18:29 ..
-rw-rw-r-- 1 ubuntu ubuntu 253 Sep 29 18:29 .gitignore
-rw-rw-r-- 1 ubuntu ubuntu 231 Sep 29 18:29 .oxlintc.json
-rw-rw-r-- 1 ubuntu ubuntu 1899 Sep 29 18:29 README.md
-rw-rw-r-- 1 ubuntu ubuntu 1396 Sep 29 18:29 index.html
-rw-rw-r-- 1 ubuntu ubuntu 108141 Sep 29 18:29 package-lock.json
-rw-rw-r-- 1 ubuntu ubuntu 797 Sep 29 18:29 package.json
-rw-rw-r-- 1 ubuntu ubuntu 80 Sep 29 18:29 postcss.config.js
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 29 18:29 public
drwxrwxr-x 8 ubuntu ubuntu 4096 Sep 29 18:29 src
-rw-rw-r-- 1 ubuntu ubuntu 5032 Sep 29 18:29 tailwind.config.js
-rw-rw-r-- 1 ubuntu ubuntu 306 Sep 29 18:29 vite.config.js
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ ~/CloudPMS-PlacementPortal/client

ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ grep 'VITE_API_BASE_URL=' .env
VITE_API_BASE_URL=/api
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ git status --short
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$
```

Fig 11.11a — Client directory with VITE_API_BASE_URL=/api configured.

```

found 0 vulnerabilities
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ npm run build

> client@0.0.0 build
> vite build

vite v8.3.1 building client environment for production...
✓ 2545 modules transformed.
computing gzip size...
dist/index.html                1.49 kB | gzip: 0.78 kB
dist/assets/index-C00-TjOp.css 50.31 kB | gzip: 8.01 kB
dist/assets/index-B5k_VLqb.js 794.08 kB | gzip: 236.55 kB

[plugin builtin:vite-reporter]
(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rollupOptions.output.codeSplitting to improve chunking: https://rollup.rs/reference/OutputOptions.codeSplitting
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
✓ built in 2.78s
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$ ls -lah dist
total 36K
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 29 19:12 .
drwxrwxr-x 6 ubuntu ubuntu 4.0K Sep 29 19:12 ..
drwxrwxr-x 2 ubuntu ubuntu 4.0K Sep 29 19:12 assets
-rw-rw-r-- 1 ubuntu ubuntu 9.3K Sep 29 19:12 favicon.svg
-rw-rw-r-- 1 ubuntu ubuntu 5.0K Sep 29 19:12 icons.svg
-rw-rw-r-- 1 ubuntu ubuntu 1.5K Sep 29 19:12 index.html
ubuntu@ip-172-31-10-206:~/CloudPMS-PlacementPortal/client$

```

Fig 11.11b — `npm run build`: 2,545 modules transformed; production bundle generated in `dist/` (JS 794 kB, 236.55 kB gzipped).

11.12 HTTPS with Let's Encrypt

A free TLS certificate for `cloudpms.kaifkhan.in` was obtained with Certbot. It is valid until 29 December 2026 and a scheduled task renews it automatically.

```

-----
No certificates found.
-----
ubuntu@ip-172-31-10-206:~$ sudo certbot certonly --nginx -d cloudpms.kaifkhan.in
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address or hit Enter to skip.
(Enter 'c' to cancel):

-----
Please read the Terms of Service at:
https://letsencrypt.org/documents/LE-SA-v1.8-July-06-2026.pdf
You must agree in order to register with the ACME server. Do you agree?
-----
(Y)es/(N)o: y
Account registered.
Requesting a certificate for cloudpms.kaifkhan.in

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/cloudpms.kaifkhan.in/fullchain.pem
Key is saved at: /etc/letsencrypt/live/cloudpms.kaifkhan.in/privkey.pem
This certificate expires on 2026-12-29.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----
ubuntu@ip-172-31-10-206:~$

```

Fig 11.12a — Certbot successfully received the certificate for `cloudpms.kaifkhan.in`.

12. Live Application Walkthrough

The deployed application is available at <https://cloudpms.kaifkhan.in>. The screens below show the main workflow of each role.

12.1 Login and Registration

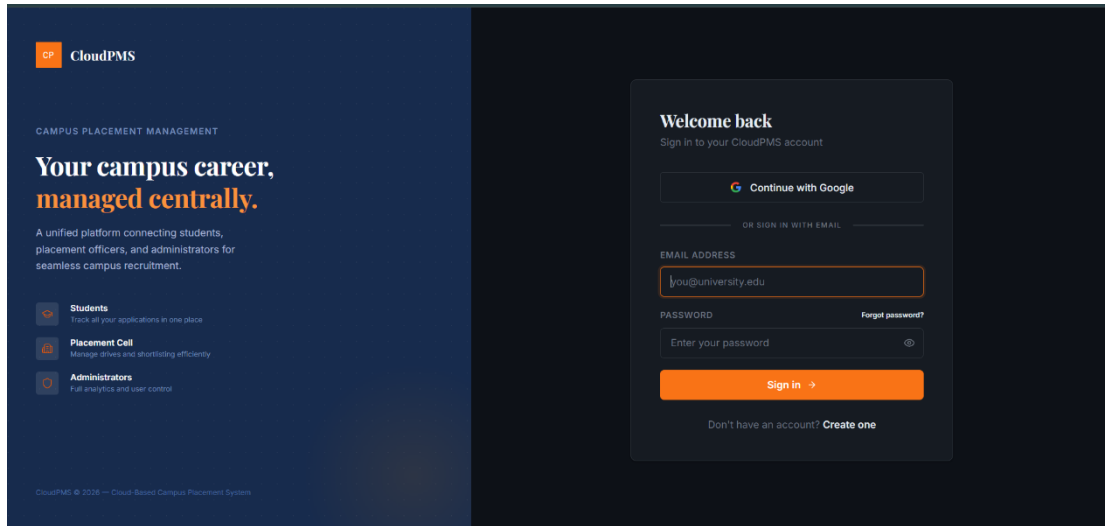
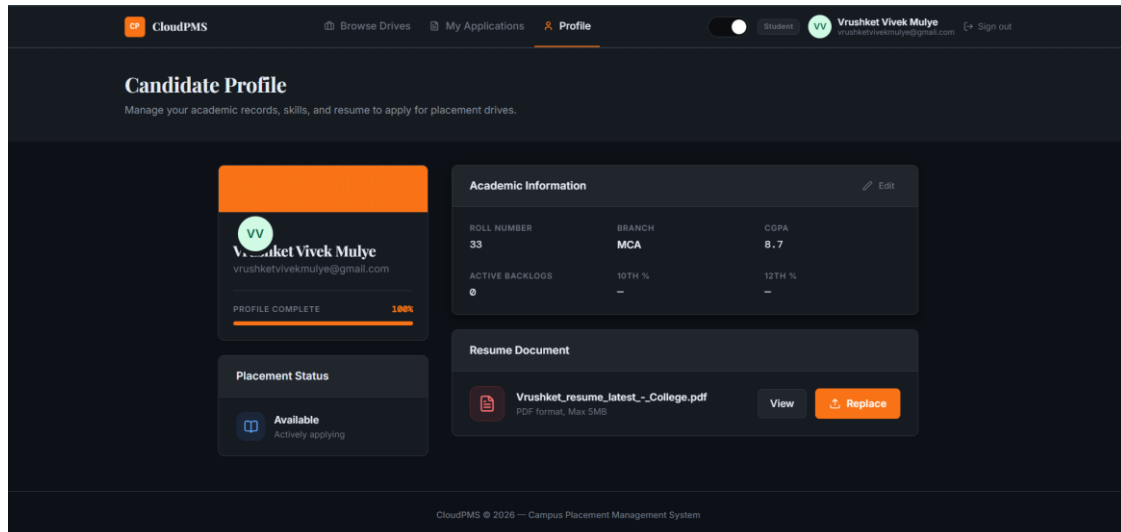


Fig 12.1a — Login screen with e-mail/password and Google sign-in.

12.2 Student Workflow



12.2a — Student profile with resume upload and skills detected by Textract.

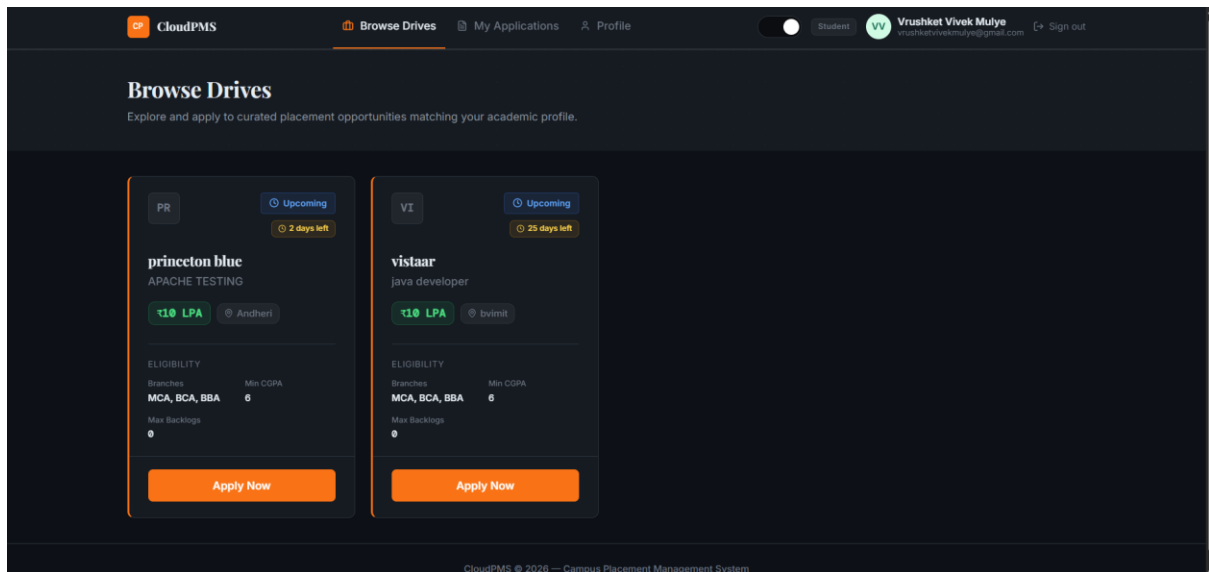


Fig 12.2b — Eligible drives list for a student.

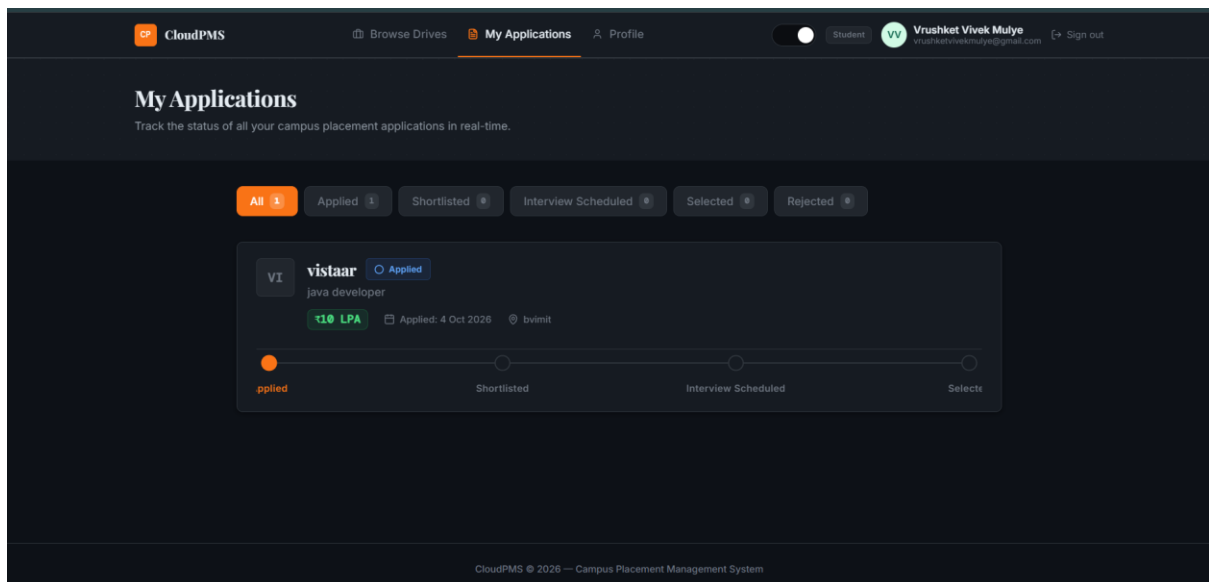


Fig 12.2c — My applications with status tracking.

12.3 Placement Cell Workflow

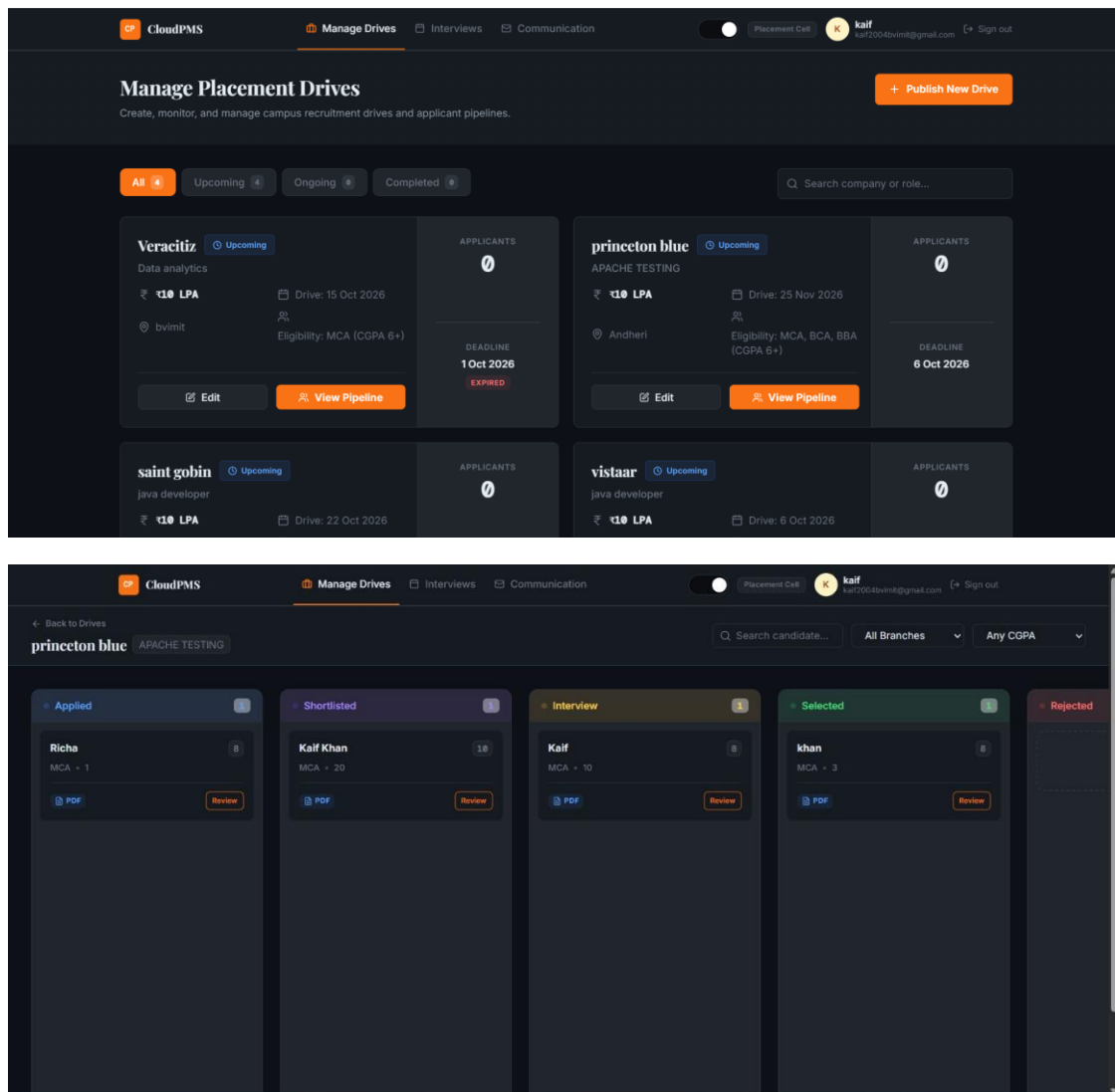


Fig 12.3a — Drive management and applicant pipeline.

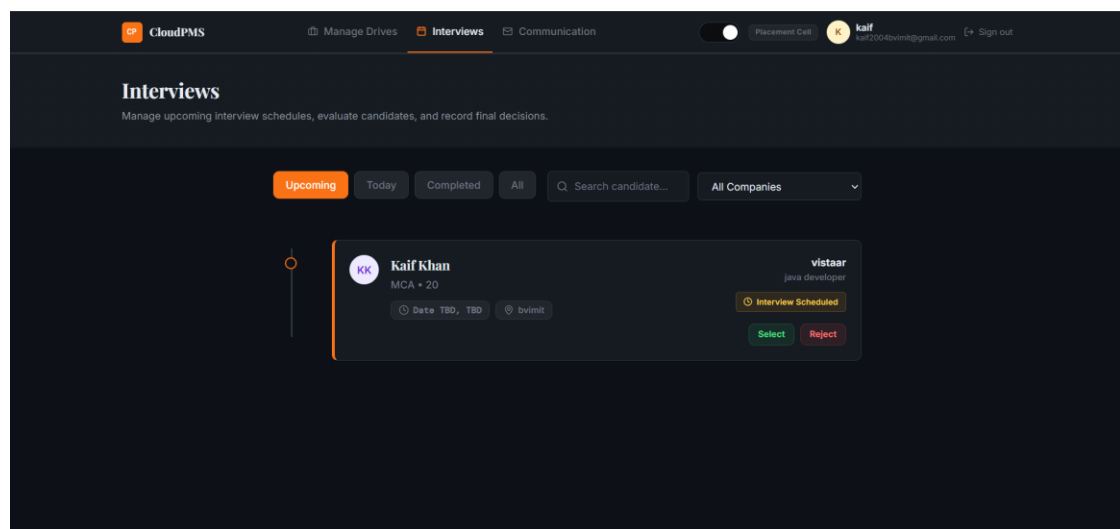


Fig 12.3b — Interview Workspace timeline.

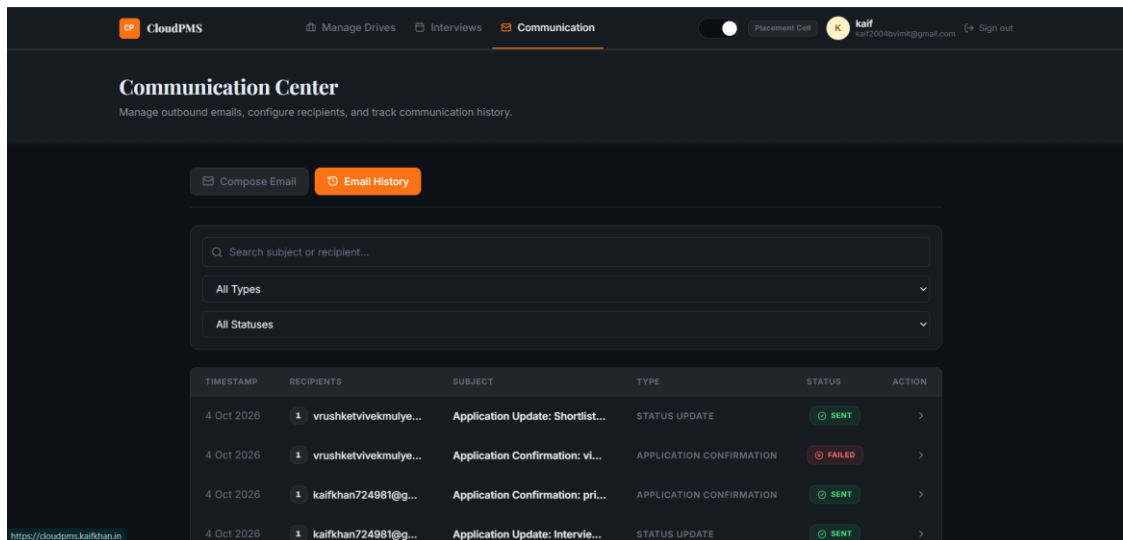


Fig 12.3c — Communication Center: compose e-mail and e-mail history.

12.4 Administrator Workflow

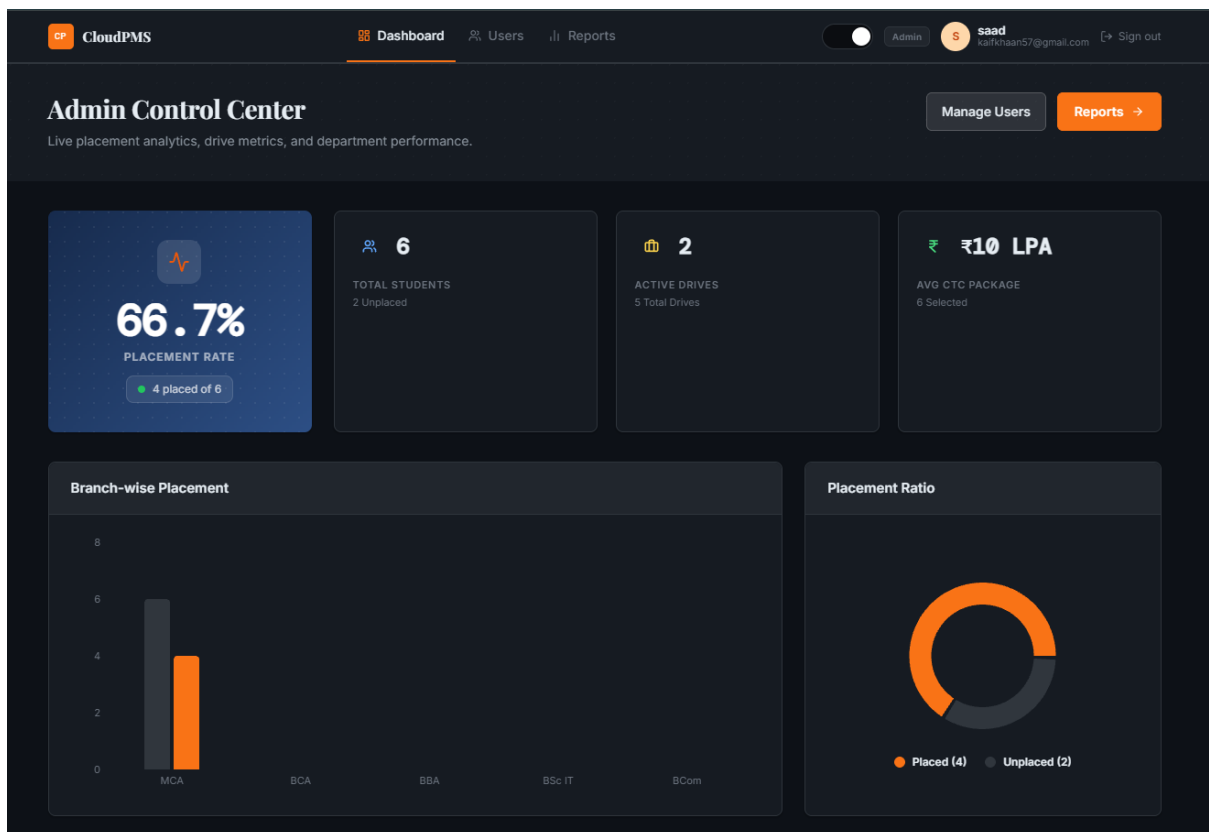


Fig 12.4a — Admin dashboard and placement reports.

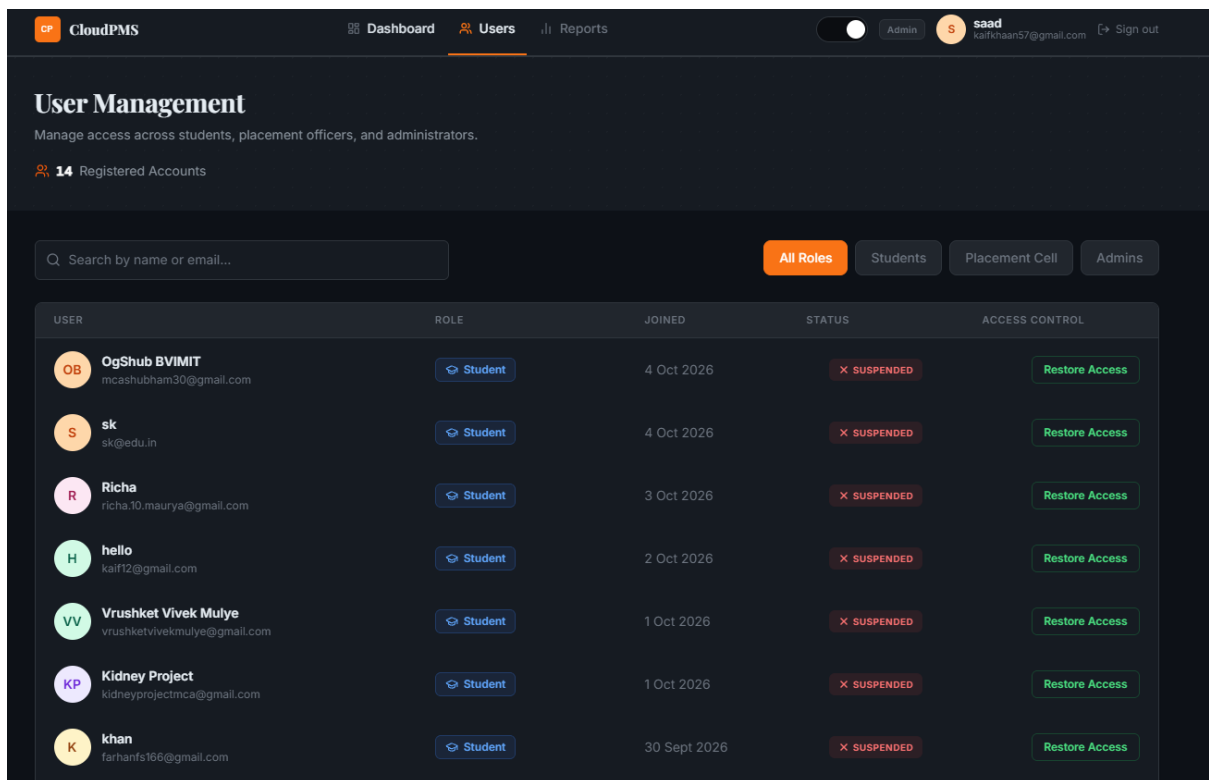


Fig 12.4b — User Management.

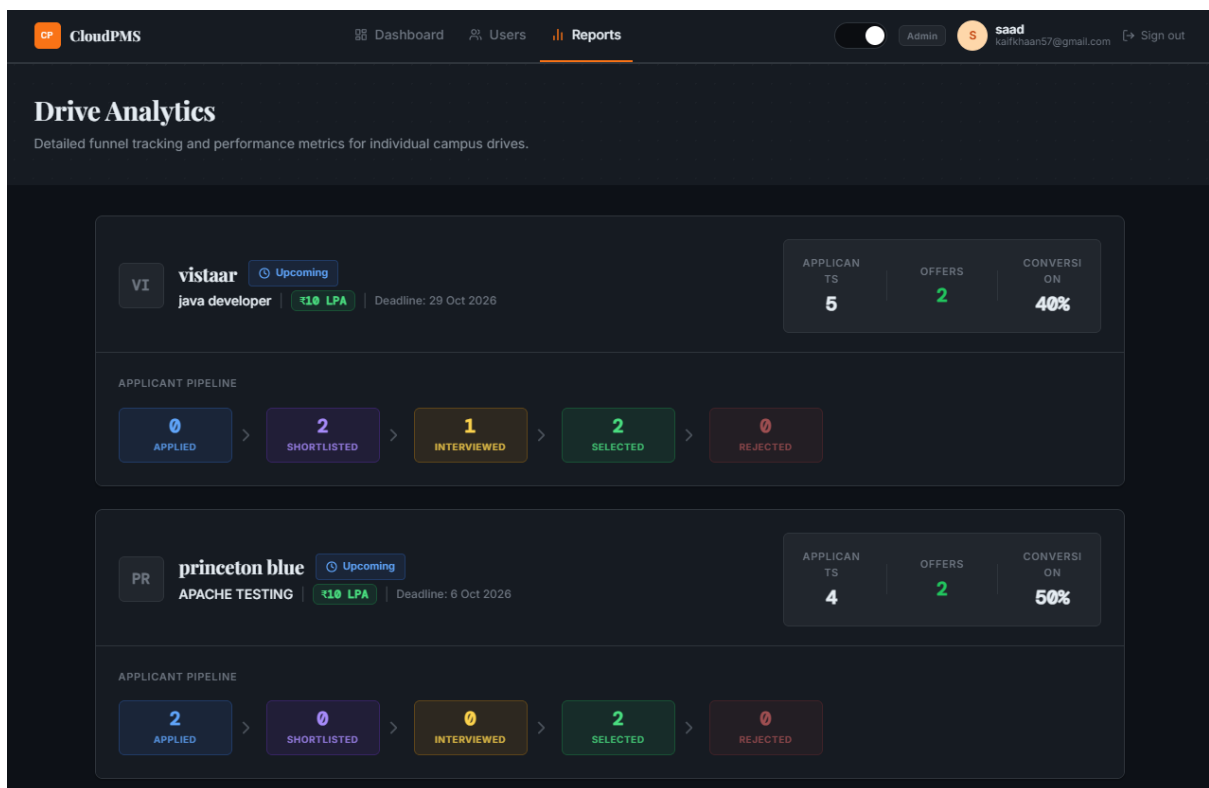


Fig 12.4c — Reports: Drive Analytics..

13. Live Deployment & Repository Links

CloudPMS is deployed on an AWS EC2 instance and is publicly accessible. The complete source code is version-controlled on GitHub.

Live Application: <https://65.1.71.208>

GitHub Repository: <https://github.com/Kaifkhan1212/CloudPMS-PlacementPortal>

Health check endpoint (server status, as verified on the instance):

```
curl http://localhost:5000/api/health
```

```
{
  "success": true,
  "statusCode": 200,
  "message": "CloudPMS API is healthy",
  "data": {
    "status": "ok",
    "aws": {
      "configured": true,
      "region": "ap-south-1",
      "s3Bucket": "cloudpms-resumes-kaif-2026",
      "textextractReady": true
    }
  }
}
```

13.1 Deployment Runbook

```
git pull origin main
cd server && npm ci && pm2 restart cloudpms && pm2 status
cd ../client && npm ci && npm run build
sudo cp -r dist/* /var/www/cloudpms/dist/
sudo nginx -t && sudo systemctl reload nginx
pm2 logs cloudpms --lines 50
```

14. Testing, Results, Limitations & Future Scope

14.1 Testing

The system was verified through manual integration testing during development and after deployment:

- **Authentication:** register → login → token refresh → logout for all three roles.
- **Resume pipeline:** PDF upload end-to-end (Multer buffer, S3 PutObject, Textract skill extraction, profile update) and presigned-URL viewing.
- **Eligibility filtering:** drive list returns only drives matching the student's branch, CGPA and backlogs.
- **Application lifecycle:** status transitions from Applied to Selected/Rejected with SES e-mail delivery and `EmailLog` entries.
- **Infrastructure:** `GET /api/health` on the instance, PM2 status, SES test e-mail, and S3 objects visible in the console.

Automated unit and integration test suites were not part of this version.

14.2 Results

- A working three-role placement platform is deployed at <https://cloudpms.kaifkhan.in> with HTTPS.
- S3, Textract and SES are integrated and operational from the EC2 instance using the IAM role.
- Nginx serves the frontend and proxies the API from a single instance; PM2 keeps the API online across crashes and reboots.
- The production frontend bundle builds in under 3 seconds and weighs 236.55 kB gzipped.

14.3 Limitations

Limitation	Detail
Single EC2 instance	No load balancing or auto scaling; instance failure takes the application offline
SES sandbox	E-mails can reach only verified addresses until production access is approved
Manual deployment	Deployments require SSH access; there is no CI/CD pipeline
Keyword-based skill extraction	A fixed dictionary is used instead of semantic NLP
Synchronous Textract call	Skill extraction runs inside the upload request and adds to its response time
No automated tests	Quality relies on manual integration testing
Bundle size	Single JavaScript chunk above 500 kB; code-splitting would improve first load

14.4 Future Scope

- Application Load Balancer and an Auto Scaling group for horizontal scaling and high availability.
- GitHub Actions CI/CD to test, build and deploy on every push.
- SES production access and domain-level DKIM sending.
- Asynchronous resume analysis with S3 events and AWS Lambda, plus semantic matching of skills.
- Admin approval workflow for new Placement Cell accounts.
- Jest and Supertest API test suites; CloudWatch metrics and alarms for observability.
- Advanced analytics: year-over-year placement trends and branch-wise placement rates.

15. Conclusion

CloudPMS demonstrates the design, development and production deployment of a cloud-based campus placement system on AWS. By combining Amazon EC2, S3, Textract, SES and IAM with MongoDB Atlas, Nginx, PM2 and Let's Encrypt, the project covers a complete real-world cloud pipeline: secure resource provisioning, identity management with instance roles, AI-powered document analysis, automated notifications and an HTTPS-enabled public endpoint.

The project deepened the team's practical understanding of IaaS and managed services, stateless service design, least-privilege security, graceful degradation and full-stack deployment on a cloud

virtual machine, and it resulted in a functional, live and publicly accessible application that addresses a real administrative need of educational institutions.

References

- Amazon Web Services documentation: EC2, S3, Textract, SES and IAM (docs.aws.amazon.com).
- MongoDB Atlas documentation (mongodb.com/docs/atlas).
- Express, React, Vite and Mongoose official documentation.
- Let's Encrypt and Certbot documentation (letsencrypt.org, certbot.eff.org).
- Project repository: github.com/Kaifkhan1212/CloudPMS-PlacementPortal.